



UNIVERSITÀ DEGLI STUDI DI SALERNO

Dipartimento di Informatica

Corso di Laurea Triennale in Informatica

TESI DI LAUREA

CNN e attacchi avversariali: trade-off accuratezza e robustezza su CIFAR-10

RELATORE

Prof. Fabio Palomba

Università degli Studi di Salerno

CANDIDATO

Rocco Vigliotta

Matricola: 0512115447

Anno Accademico 2025-2026

Questa tesi è stata realizzata nel

sesa^{lab}
SOFTWARE ENGINEERING
SALERNO

“A volte la realtà supera la fantasia.”

Detto popolare (di ascendenza pirandelliana)

Abstract

I modelli di deep learning hanno raggiunto prestazioni eccezionali nella classificazione di immagini, ma sono vulnerabili ad attacchi avversariali: perturbazioni minime e spesso impercettibili che inducono errori di classificazione catastrofici. Questa tesi analizza empiricamente il fenomeno degli attacchi avversariali e l'efficacia dell'adversarial training come tecnica di difesa, utilizzando il dataset CIFAR-10 come benchmark.

È stata implementata una rete neurale convoluzionale custom con 590,794 parametri, addestrata in tre configurazioni: un modello baseline standard e due modelli robusti tramite adversarial training con FGSM (Fast Gradient Sign Method) e PGD (Projected Gradient Descent). I modelli sono stati valutati su dati puliti e sotto attacco con budget di perturbazione $\epsilon = 0.03$.

I risultati evidenziano un trade-off significativo tra accuratezza e robustezza: il modello baseline raggiunge l'86.91% di accuratezza su dati puliti ma crolla all'1.41% (FGSM) e allo 0.0% (PGD) sotto attacco. I modelli adversarially trained mantengono un'accuratezza del 79% sui dati puliti e una robustezza di circa il 40% (FGSM) e il 34% (PGD) sotto attacco, confermando l'efficacia dell'adversarial training al costo di 7-8 punti percentuali di accuratezza standard. Il modello addestrato con PGD dimostra robustezza superiore rispetto a quello addestrato con FGSM, coerentemente con quanto riportato in letteratura.

Questo lavoro fornisce un'analisi sperimentale rigorosa del compromesso accuratezza-robustezza, offrendo spunti per lo sviluppo di sistemi di machine learning più sicuri.

Indice

Elenco delle Figure	v
Elenco delle Tabelle	vii
1 Introduzione	1
1.1 Contesto Applicativo	1
1.2 Motivazioni e Obiettivi	4
1.2.1 Domande di Ricerca	4
1.2.2 Obiettivi Operativi	4
1.3 Risultati Ottenuti	6
1.4 Struttura della tesi	7
2 Background e Stato dell'Arte	9
2.1 Reti Neurali Convoluzionali (CNN)	9
2.1.1 Convolutional Layers: il cuore della CNN	10
2.1.2 Pooling Layers: riduzione dimensionale e robustezza locale .	11
2.1.3 Fully Connected Layers: il classificatore finale	12
2.1.4 Softmax e Cross-Entropy Loss	12
2.1.5 Backpropagation e Gradient Descent	13
2.1.6 Activation Functions: non-linearità essenziali	14
2.2 Architettura del Modello e Scelte di Design	15

2.2.1	Evoluzione delle architetture CNN	15
2.2.2	Batch Normalization: stabilità e velocità	16
2.2.3	Dropout: regolarizzazione contro l'overfitting	18
2.2.4	Adaptive Average Pooling: flessibilità nell'output	18
2.3	Dataset CIFAR-10	19
2.3.1	Struttura e caratteristiche del dataset	19
2.3.2	Perché CIFAR-10 è un benchmark importante	20
2.4	Adversarial Machine Learning: Definizioni e Concetti Fondamentali	22
2.4.1	Cos'è un esempio avversariale?	22
2.4.2	Norme di perturbazione: ℓ_∞ , ℓ_2 , ℓ_1 , ℓ_0	23
2.4.3	Threat Model: white-box vs black-box	24
2.4.4	Budget di perturbazione (ϵ)	25
2.4.5	Transferability: la vulnerabilità condivisa	25
2.4.6	Perché esistono gli esempi avversariali?	26
2.5	Attacchi Adversariali	27
2.5.1	Fast Gradient Sign Method (FGSM)	27
2.5.2	Projected Gradient Descent (PGD)	29
2.5.3	Altri Attacchi Rilevanti (cenni)	30
2.6	Tecniche di Difesa	32
2.6.1	Adversarial Training	32
2.6.2	Altre Difese	34
2.7	Il Trade-off Accuratezza-Robustezza	36
2.7.1	Quantificare il trade-off	38
2.7.2	Implicazioni pratiche	39
2.8	Metriche di Valutazione	41
2.8.1	Metriche Standard	41
2.8.2	Metriche per Adversarial Robustness	43
3	Metodo di Ricerca	45
3.1	Domande di Ricerca e Design Sperimentale	45
3.2	Panoramica della Metodologia	47
3.3	Setup Sperimentale	48

3.3.1	Hardware e Ambiente di Calcolo	48
3.3.2	Librerie e Dipendenze Software	49
3.4	Pipeline di Dati e Preprocessing	50
3.4.1	Acquisizione e Caching del Dataset	50
3.4.2	Preprocessing e Normalizzazione	51
3.4.3	Strategia di Data Augmentation	51
3.5	Architettura del Modello	52
3.5.1	Progettazione e Filosofia Architetturale	52
3.5.2	Descrizione Stratificata dell'Architettura	52
3.5.3	Analisi Quantitativa del Modello	53
3.5.4	Motivazione delle Scelte Architetturali	54
3.6	Training dei Modelli	54
3.6.1	Modello Baseline (Clean)	54
3.6.2	Adversarial Training con FGSM	56
3.6.3	Adversarial Training con PGD	56
3.7	Implementazione degli Attacchi Adversariali	57
3.7.1	Fast Gradient Sign Method (FGSM)	57
3.7.2	Projected Gradient Descent (PGD)	58
3.8	Pipeline di Valutazione	58
3.8.1	Architettura del Sistema di Valutazione	59
3.8.2	Protocollo di Testing e Metriche	59
3.8.3	Visualizzazione dei Risultati e Analisi di Interpretabilità	60
3.9	Riproducibilità	61
3.9.1	Gestione Deterministica della Stocasticità	61
3.9.2	Checkpointing, Logging e Configurazione	61
3.9.3	Limitazioni e Raccomandazioni per la Replica	62
4	Analisi dei Risultati	64
4.1	Prestazioni del Modello Clean	64
4.1.1	Accuratezza su Dati Puliti	64
4.1.2	Vulnerabilità sotto Attacco Adversariale	66
4.1.3	Visualizzazione di Esempi Adversariali	70

4.2	Efficacia dell'Adversarial Training	72
4.2.1	Prestazioni Complessive	72
4.2.2	Analisi delle Confusion Matrix	73
4.2.3	Metriche di Robustezza Avanzate	74
4.3	Analisi del Trade-off Accuratezza-Robustezza	75
4.3.1	Quantificazione del Trade-off e Robustness Gap	75
4.3.2	Esiste un "Limite Fondamentale"?	77
4.4	Confronto FGSM Training vs PGD Training	78
4.4.1	Confronto delle Prestazioni e Valutazione Incrociata	79
4.5	Costo Computazionale e Implicazioni Pratiche	81
4.5.1	Tempi di Training e Valutazione	81
4.5.2	Memoria e Risorse Hardware	82
4.5.3	Implicazioni per il Deployment	83
4.6	Analisi Qualitativa e Interpretabilità	84
4.7	Risultati e Risposte: Il Quadro Completo	87
5	Conclusioni: Verso sistemi di ML robusti e affidabili	88
5.1	Risposte alle Domande di Ricerca	89
5.2	Implicazioni Pratiche	90
5.3	Limitazioni dello Studio	90
5.4	Riflessioni Finali	91
	Bibliografia	93

Elenco delle figure

1.1	Esempio di adversarial example in ambito medico	2
1.2	Timeline evolutiva degli attacchi adversariali	3
2.1	Esempio schematico dell'operazione di convoluzione in una CNN . .	11
2.2	Confronto tra una CNN addestrata con e senza Batch Normalization.	17
2.3	Esempi rappresentativi delle dieci classi del dataset CIFAR-10.	20
2.4	Proiezione 2D del loss landscape in uno spazio di input ad alta dimensionalità.	28
3.1	Panoramica del progetto	48
4.1	Curva di training e validation loss del modello clean.	65
4.2	Confusion matrix del modello clean su dati puliti (valori assoluti su 1000 esempi per classe).	66
4.3	Confusion matrix del modello clean sotto attacco FGSM ($\epsilon = 0.03$). .	68
4.4	Distribuzione dei confidence score per il modello clean su dati puliti, sotto attacco FGSM e sotto attacco PGD.	69
4.5	Esempi di immagini originali (prima colonna) e le corrispondenti versioni perturbate con FGSM (seconda colonna) e PGD (terza colonna). Per ogni immagine è indicata la vera classe, la classe predetta e il confidence score.	71

4.6	Confronto delle confusion matrix per tre modelli sotto attacco PGD ($\epsilon = 0.03$).	73
4.7	Confronto del Robustness Score e dell'Attack Success Rate	77
4.8	Analisi di interpretabilità su un campione di classe "cat".	85
4.9	Distribuzione delle confidence scores (box plot).	86

Elenco delle tabelle

2.1	Confronto tra FGSM e PGD rispetto a vari aspetti	30
2.2	Confronto delle principali tecniche di difesa adversarial	36
2.3	Trade-off accettabili in diversi contesti applicativi	39
3.1	Panoramica delle principali dipendenze software e delle loro versioni.	49
4.1	Prestazioni del modello in condizioni clean e sotto attacco avversario.	67
4.2	Sintesi delle prestazioni e della robustezza dei modelli valutati. . . .	72
4.3	Fooling rate, robustezza e distorsione L_2 dei modelli sotto attacchi FGSM e PGD.	74
4.4	Analisi quantitativa del trade-off accuratezza-robustezza.	75
4.5	Confronto prestazioni FGSM-trained vs PGD-trained	79
4.6	Confronto costo-beneficio FGSM vs PGD training.	80
4.7	Overhead computazionale dell'adversarial training e della valutazione sotto attacco. I tempi sono misurati su GPU T4.	82
4.8	Utilizzo di memoria GPU (batch size = 128) e batch size massimo su T4 (16GB).	82
4.9	Sintesi completa dei risultati sperimentali.	87

CAPITOLO 1

Introduzione

1.1 Contesto Applicativo

Immaginate un'auto a guida autonoma che, di fronte a un segnale di stop leggermente alterato da un adesivo, lo interpreti erroneamente come un limite di velocità [1]. Oppure un sistema di diagnostica medica che, analizzando una radiografia contenente un tumore ma modificata da un artefatto digitale impercettibile, fornisca con alta confidenza un responso di perfetta sanità. Un esempio di questa vulnerabilità è illustrato nella Figura 1.1.

Questi non sono scenari di fantascienza, ma vulnerabilità concrete e allarmanti che emergono dall'intersezione tra intelligenza artificiale e sicurezza.

Negli ultimi anni, l'avvento delle reti neurali profonde, e in particolare delle Convolutional Neural Networks (CNN), ha davvero rivoluzionato il campo della visione artificiale [2].

Questi modelli, ispirati al sistema visivo biologico, hanno imparato a "vedere" in modo sorprendentemente efficace, superando in molti compiti specifici le capacità umane. La loro abilità di estrarre automaticamente pattern complessi dai pixel le ha rese il motore di innumerevoli applicazioni: non solo nei sistemi critici come quelli

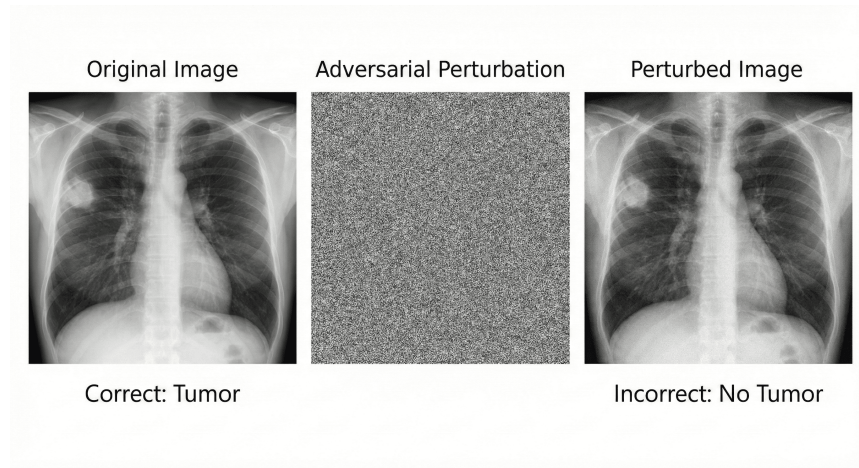


Figura 1.1: Esempio di adversarial example in ambito medico

menzionati, ma anche nei filtri dei social media, nei suggerimenti di acquisto e negli assistenti virtuali. La loro pervasività, tuttavia, le rende un bersaglio ideale.

Per addestrare e testare questi "occhi artificiali", la comunità scientifica si affida a benchmark come il dataset CIFAR-10 [3].

Con le sue 60.000 immagini in miniatura (32x32 pixel) che ritraggono aerei, automobili, uccelli e altre categorie, CIFAR-10 è diventato una palestra fondamentale. La sua apparente semplicità nasconde in realtà una sfida non banale: insegnare a un modello a riconoscere e discernere un gatto da un cane, un cervo da un cavallo, nonostante la bassa risoluzione e l'enorme varietà di pose e sfondi. Un modello che performa bene qui ha imparato a generalizzare ed astrarre concetti visivi, un'abilità che crediamo sia solida ed affidabile.

Eppure, questa fiducia è stata scossa nel 2014 da una scoperta sconcertante. Szegedy et al. [4] hanno dimostrato che queste reti neurali, così potenti, possiedono un tallone d'Achille: sono incredibilmente sensibili a perturbazioni minuscole, rumore digitale aggiunto all'immagine che è spesso invisibile all'occhio umano, ma che può portare la rete a commettere errori catastrofici. È l'equivalente digitale di un'illusione ottica: un panda che, con l'aggiunta di un rumore calcolato scientificamente, viene visto dalla rete come un gibbone. Questo fenomeno ha un nome: attacco avversariale.

La domanda che sorge spontanea è inquietante: se un modello può essere ingannato così facilmente, possiamo davvero fidarci delle sue decisioni in contesti dove la posta in gioco è la vita umana o la sicurezza nazionale? Gli adversarial attacks non so-

no più una semplice curiosità accademica; sono una minaccia tangibile all'affidabilità dell'IA.

La corsa agli armamenti è così iniziata. Da un lato, gli attaccanti hanno affinato le loro tecniche. Il Fast Gradient Sign Method (FGSM), proposto da Goodfellow et al. [4] nel 2015, è un attacco rapido ed efficiente che sfrutta la direzione del gradiente della rete per creare perturbazioni in un solo passo. Dall'altro lato, il Projected Gradient Descent (PGD), introdotto da Madry et al. [5], rappresenta l'assedio metodico: un attacco iterativo e più potente, considerato per anni il gold standard, ma come mostrato in figura 1.2, l'evoluzione degli attacchi avversariali segue una progressione crescente in efficacia e complessità.

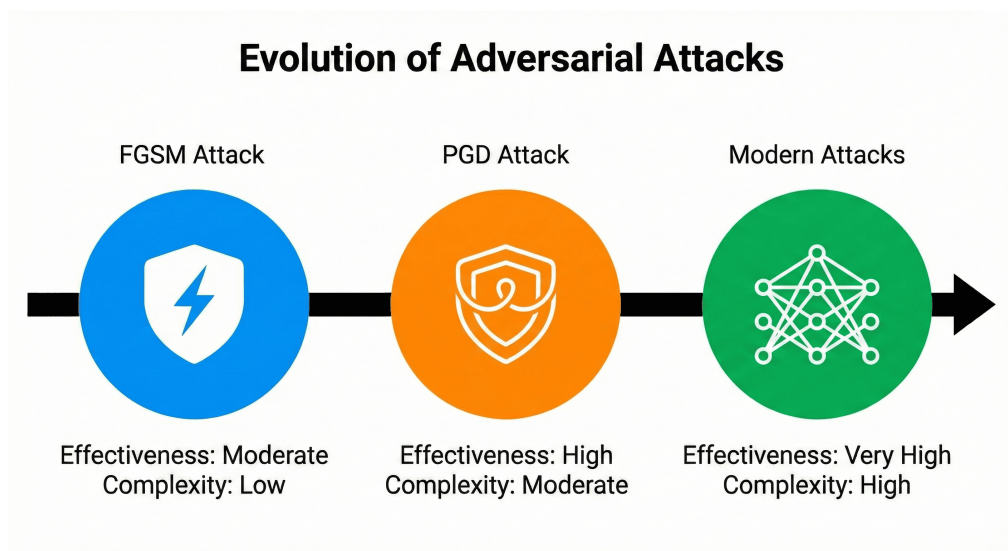


Figura 1.2: Timeline evolutiva degli attacchi avversariali

Dall'altra parte della barricata, i difensori hanno sviluppato contromisure. Ci sono validi esempi, come l'adversarial training, una sorta di vaccinazione per le reti neurali. Il principio è semplice, ma efficace: si addestra il modello non solo con immagini pulite, ma anche con esempi avversari, costringendolo a imparare a resistere a tali manipolazioni. Tuttavia, come spesso accade in ingegneria, non esiste un pranzo gratis. Questa robustezza ha un costo: i modelli "vaccinati" tendono a perdere un po' della loro acutezza visiva sui dati originali, non perturbati. Si crea così un trade-off fondamentale, un delicato equilibrio tra l'essere accurati in un mondo ideale e l'essere robusti in uno ostile [5]. Comprendere e quantificare questo compromesso è la sfida centrale che questo lavoro di tesi intende esplorare.

1.2 Motivazioni e Obiettivi

1.2.1 Domande di Ricerca

Questo lavoro si propone di rispondere alle seguenti domande di ricerca:

Q RQ₁. *Quanto sono vulnerabili le reti neurali convoluzionali standard agli attacchi avversariali?*

In particolare, si vuole quantificare il degrado delle prestazioni di un modello CNN non protetto quando sottoposto ad attacchi FGSM e PGD.

Q RQ₂. *L'adversarial training è una difesa efficace contro gli attacchi avversariali?*

Si intende valutare se l'esposizione a esempi avversariali durante il training aumenta significativamente la robustezza del modello e in quale misura.

Q RQ₃. *Esiste un trade-off tra accuratezza standard e robustezza avversariale?*

Si vuole quantificare la perdita di accuratezza sui dati puliti necessaria per ottenere robustezza, analizzando se questo compromesso è inevitabile o può essere mitigato.

Q RQ₄. *PGD training conferisce maggiore robustezza rispetto a FGSM training?*

Si intende confrontare l'efficacia delle due strategie di adversarial training, valutando se l'uso di un attacco più forte durante il training si traduce in maggiore robustezza generalizzata.

Q RQ₅. *Qual è il costo computazionale dell'adversarial training?*

Si vuole misurare l'overhead¹ temporale e computazionale richiesto dall'adversarial training rispetto al training standard, per valutarne la fattibilità pratica.

Queste domande di ricerca guidano il design sperimentale descritto nel Capitolo 3 e forniscono il framework per l'analisi dei risultati presentata nel Capitolo 4.

1.2.2 Obiettivi Operativi

Di fronte a un panorama letterario ricchissimo di algoritmi sempre più sofisticati, un ricercatore o un praticante che si avvicini per la prima volta all'Adversarial Machine Learning potrebbe sentirsi sopraffatto [6]. La teoria a volte rischia di offuscare la sostanza pratica [7].

¹Per overhead si intende il costo aggiuntivo in tempo o risorse rispetto alla base.

Questa tesi nasce proprio dal desiderio di colmare questo gap tra l'astrazione teorica e la realtà sperimentale. Non si vuole inventare una nuova tecnica rivoluzionaria, ma piuttosto fornire una mappatura chiara, empirica e riproducibile di un territorio complesso, utilizzando strumenti noti per costruire una comprensione profonda.

© **Our Goal.** L'obiettivo principale di questa tesi è quantificare empiricamente il trade-off tra accuratezza standard e robustezza avversariale in reti neurali convoluzionali, confrontando l'efficacia di due strategie di adversarial training (FGSM e PGD) su CIFAR-10.

Il percorso di ricerca che abbiamo disegnato si articola in cinque tappe fondamentali:

1. **Costruire una Base Solida:** Implementare da zero un'architettura CNN custom per CIFAR-10, bilanciando complessità ed efficienza. Il nostro modello, pur nella sua semplicità, è stato progettato per raggiungere prestazioni competitive (circa l'87% di accuratezza), fornendo così un punto di partenza credibile e un soggetto sano per i nostri esperimenti.
2. **Sferrare l'Attacco:** Mettere alla prova il nostro modello vergine con due attacchi rappresentativi: il FGSM, per testare una minaccia rapida e semplice, e il PGD, per simulare un avversario determinato e metodico. L'obiettivo è misurare con precisione l'entità della vulnerabilità in scenari diversi.
3. **Organizzare la Difesa:** Addestrare due nuovi modelli attraverso l'adversarial training, uno vaccinato contro il FGSM e l'altro, più robusto, contro il PGD. Questa fase ci permetterà di osservare come una rete neurale impara a difendersi quando esposta strategicamente al pericolo.
4. **Il Confronto Finale:** Valutare i tre modelli – il modello standard, il "FGSM-trained" e il "PGD-trained" – su un triplice fronte: dati puliti, attacco FGSM e attacco PGD. Questo confronto incrociato è la chiave per svelare le dinamiche di robustezza e i loro costi nascosti.
5. **Analizzare il Compromesso:** Andare oltre i singoli numeri per quantificare e visualizzare il trade-off tra accuratezza e robustezza. Questo non è un mero

esercizio accademico, ma una riflessione con implicazioni pratiche: in un mondo reale, è meglio un modello super-preciso ma manipolabile, o uno leggermente meno accurato, ma su cui possiamo contare anche sotto attacco?

Il valore di questo lavoro risiede proprio in questo approccio didattico e sistematico. È un invito a sporcarsi le mani con il codice e i dati, per costruire un'intuizione pratica di un fenomeno che, se rimane confinato sulle pagine dei paper, rischia di restare astratto. Capire concretamente quanto sia fragile un modello e quanto sia difficile proteggerlo è una lezione fondamentale per chiunque voglia progettare sistemi di IA non solo intelligenti, ma anche sicuri e affidabili.

1.3 Risultati Ottenuti

I nostri esperimenti hanno dipinto un quadro netto che conferma la teoria e le dà un volto concreto e quantificabile [4, 5].

Il nostro modello "clean", addestrato per 20 epoche, ha raggiunto un rispettabile 86.91% di accuratezza sul test set originale. Un osservatore esterno lo giudicherebbe un modello ben funzionante e competente. Tuttavia, questo castello di carte è crollato alla prima prova di forza. Sotto l'attacco FGSM (con un budget di perturbazione $\epsilon=0.03$), la sua accuratezza è precipitata a un misero 1.41%. Ancora più drammatico è stato l'esito contro il PGD: uno 0.0% schiacciante. Il modello, così abile nel distinguere un cane da un gatto in condizioni normali, è stato ingannato sistematicamente e senza appello. È la dimostrazione più chiara che la performance su dati puliti è una metrica gravemente insufficiente per valutare l'affidabilità di un sistema di IA.

La risposta difensiva dell'adversarial training ha dato risultati significativi e illuminanti. Il modello addestrato con FGSM ha pagato il suo biglietto per la robustezza: la sua accuratezza su dati puliti è scesa a 79.39%, un calo di circa 7 punti percentuali. Tuttavia, quando attaccato con il FGSM, ha mantenuto un'accuratezza di 39.01% – un miglioramento di oltre 37 punti rispetto al modello clean. Anche contro il PGD, più forte, ha tenuto testa con un 30.64%, un risultato comunque infinitamente superiore allo 0% di partenza.

Il modello addestrato con PGD ha mostrato il valore di una preparazione più rigorosa. Pur partendo da un'accuratezza su dati puliti simile (78.98%), ha dimostrato una robustezza superiore e più generalizzata, raggiungendo 40.75% sotto FGSM e 34.03% sotto PGD. Questo conferma un principio chiave: addestrare contro l'attaccante più forte disponibile fornisce una protezione più solida e versatile [5].

Il trade-off emerge quindi in modo cristallino. Guadagnare una robustezza non banale contro attacchi sofisticati costa circa 7-8 punti percentuali di accuratezza standard. La domanda cruciale diventa: quando vale la pena di accettare questo compromesso? La risposta non è tecnica, ma strategica ed etica. In un'applicazione di intrattenimento, forse no. Ma in uno scenario in cui un errore potrebbe avere conseguenze irreversibili, come un'auto che ignora un segnale di stop o una diagnosi medica errata, sacrificare una piccola parte di precisione per guadagnare una grande dose di sicurezza non è solo una scelta sensata: è un imperativo.

1.4 Struttura della tesi

Per accompagnare il lettore in questo viaggio esplorativo, la tesi è organizzata in cinque capitoli (incluso il capitolo introduttivo), ciascuno con una missione precisa:

- **Capitolo 2 – Background e Stato dell'Arte:** Getteremo le basi teoriche, introducendo i principi delle CNN e del Machine Learning Avversario. Faremo una rassegna delle principali tecniche di attacco come FGSM e PGD, e di difesa, per inquadrare il nostro lavoro nel contesto scientifico più ampio.
- **Capitolo 3 – Metodo di Ricerca:** Questo è il cuore operativo della tesi. Descriveremo nel dettaglio l'architettura della CNN da noi progettata, i parametri di addestramento, la logica dietro l'implementazione degli attacchi FGSM e PGD, e l'intera pipeline sperimentale. L'obiettivo è garantire la massima trasparenza e riproducibilità.
- **Capitolo 4 – Analisi dei Risultati:** Qui i dati parlano. Presenteremo i risultati quantitativi attraverso tabelle e grafici, per confrontare in modo rigoroso le performance dei tre modelli. Arricchiremo l'analisi con visualizzazioni delle

immagini perturbate e delle classificazioni errate, per dare un volto concreto ai numeri. L'analisi del trade-off accuratezza-robustezza sarà il fulcro di questo capitolo.

- **Capitolo 5 – Conclusioni:** Tireremo le somme del lavoro, riassumendo i contributi principali e le lezioni apprese. Discuteremo apertamente i limiti del nostro approccio sperimentale e delinearemo possibili strade per future esplorazioni, lasciando al lettore non solo una risposta, ma anche nuove domande su cui riflettere.

Background e Stato dell'Arte

2.1 Reti Neurali Convoluzionali (CNN)

Le Convolutional Neural Networks si sono affermate come l'approccio predominante nell'elaborazione di immagini [3, 8], dimostrando prestazioni eccezionali in numerosi ambiti applicativi. Ciò che le distingue, tuttavia, non è solo la loro efficacia, ma anche una peculiare dualità: la stessa struttura che garantisce risultati così avanzati le rende al contempo sensibili a manipolazioni sottili. Comprendere i meccanismi architetturali che governano il loro funzionamento permette di illuminare questa apparente contraddizione [4]. A differenza delle reti fully-connected, che elaborano l'intero input in modo indifferenziato, le CNN incorporano un principio organizzativo fondamentale: l'informazione visiva possiede una struttura spaziale gerarchica, dove i pattern locali costituiscono i mattoni fondamentali della rappresentazione. I pixel vicini tra loro tendono a essere correlati e a formare pattern locali come bordi, texture, forme, che si ripetono in diverse parti dell'immagine.

2.1.1 Convolutional Layers: il cuore della CNN

Il layer convoluzionale è l'elemento distintivo di queste architetture. Invece di applicare una trasformazione lineare completa, la convoluzione utilizza filtri (o kernel) di piccole dimensioni, tipicamente 3×3 o 5×5 , che "scorrono" sull'immagine di input, calcolando il prodotto scalare tra il filtro e la porzione di immagine sottostante. Questo meccanismo permette di rilevare caratteristiche locali come bordi, angoli, o texture, indipendentemente dalla loro posizione nell'immagine. Matematicamente, l'operazione di convoluzione 2D può essere espressa come:

Convoluzione 2D:

$$(f * g)(i, j) = \sum_m \sum_n f(m, n) \cdot g(i - m, j - n)$$

- f : Immagine di input (feature map¹ del layer precedente).
- g : Kernel o filtro della convoluzione.
- (i, j) : Coordinate della feature map di output.
- (m, n) : Coordinate all'interno del kernel g .
- $\sum_m \sum_n$: Doppia sommatoria che effettua il prodotto scalare tra il kernel e la regione dell'immagine sottostante.

Ogni filtro apprende automaticamente durante il training a riconoscere un particolare tipo di pattern. Nei primi layer della rete, i filtri tendono a catturare caratteristiche semplici e a basso livello come linee verticali, orizzontali, diagonali, sfumature di colore. Man mano che si procede in profondità nella rete, i filtri imparano a riconoscere pattern sempre più complessi e astratti [9]: nei layer intermedi emergono forme geometriche, texture, parti di oggetti (come occhi, ruote, ali); nei layer finali si formano rappresentazioni ad alto livello che corrispondono a concetti semantici completi. Un aspetto cruciale delle convoluzioni è il *parameter sharing*: lo stesso filtro viene applicato a tutta l'immagine, riducendo drasticamente il numero di parametri

¹E' la rappresentazione bidimensionale dell'attivazione di un filtro convoluzionale applicato all'input, che evidenzia la presenza di specifiche caratteristiche locali dell'immagine.

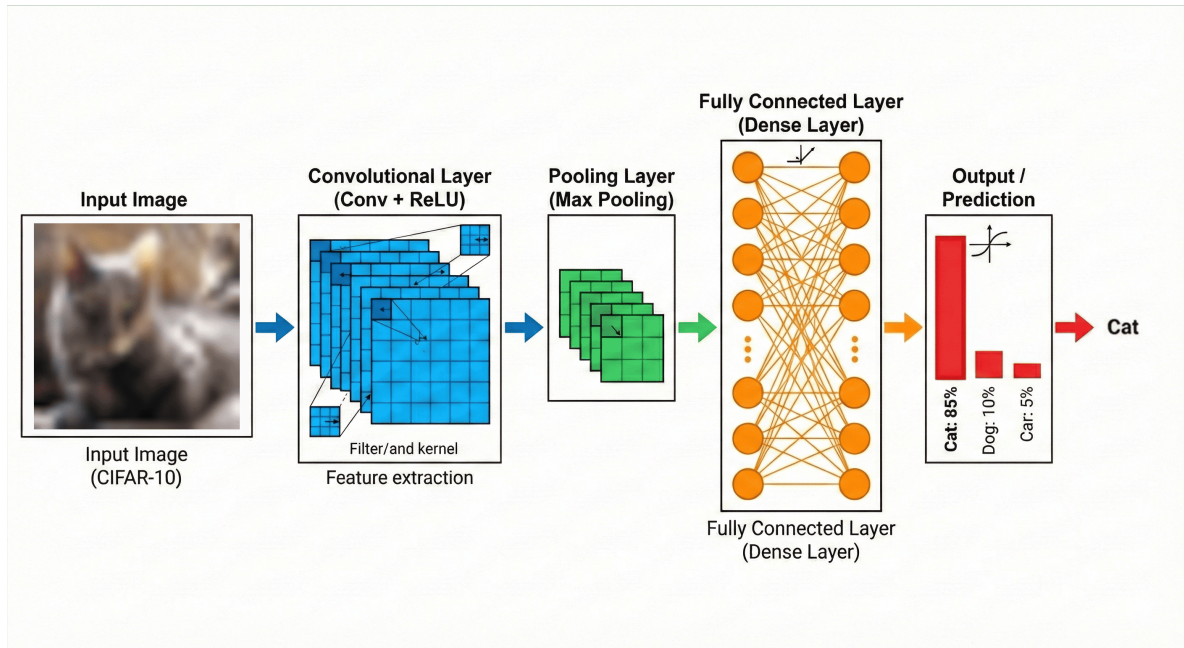


Figura 2.1: Esempio schematico dell'operazione di convoluzione in una CNN

da apprendere rispetto a una rete fully-connected. Questo non solo rende il training più efficiente, ma introduce anche un bias induttivo importante: la rete assume che le caratteristiche utili possano apparire in qualsiasi parte dell'immagine, una proprietà nota come *translation invariance*.

2.1.2 Pooling Layers: riduzione dimensionale e robustezza locale

Dopo i layer convoluzionali, è comune inserire layer di pooling, che riducono progressivamente la dimensionalità spaziale delle feature map. Tra le operazioni di pooling, il max pooling rappresenta la scelta più consolidata. Questo processo organizza l'input in regioni distinte e conserva esclusivamente il valore più elevato presente in ciascuna di esse. Tale meccanismo produce due vantaggi fondamentali: da un lato, opera una riduzione dimensionale sistematica, contenendo la complessità computazionale; dall'altro, conferisce una certa insensibilità a lievi variazioni posizionali, preservando l'informazione più saliente anche in presenza di piccole discrepanze nell'input. Se un bordo si sposta leggermente, il max pooling tenderà comunque a catturare il segnale più forte nella regione, mantenendo stabile la rappresentazione. Tuttavia, questa riduzione dimensionale comporta anche una perdita di informazione spaziale precisa [10]. Negli anni, sono state proposte alternative al

max pooling, come l'average pooling (che calcola la media invece del massimo) o, più recentemente, l'adaptive average pooling, che permette di ottenere un output di dimensione fissa indipendentemente dalla dimensione dell'input.

2.1.3 Fully Connected Layers: il classificatore finale

Dopo una serie di layer convoluzionali e di pooling, le feature map vengono "appiattite" in un vettore unidimensionale e passate a uno o più fully connected layers (o dense layers), che fungono da classificatore. Questi layer combinano le caratteristiche estratte dalle convoluzioni per produrre la predizione finale. L'ultimo layer fully-connected ha tipicamente tanti neuroni quante sono le classi del problema (10 nel caso di CIFAR-10), e l'output viene interpretato come un vettore di logits², punteggi grezzi non normalizzati che rappresentano la "confidenza" della rete per ciascuna classe, prima di qualsiasi normalizzazione probabilistica.

2.1.4 Softmax e Cross-Entropy Loss

La conversione dei logits in probabilità avviene mediante la funzione softmax, che normalizza i punteggi grezzi trasformandoli in una distribuzione di probabilità coerente:

Funzione Softmax:

$$\text{softmax}(z_i) = \frac{\exp(z_i)}{\sum_{j=1}^K \exp(z_j)}$$

- z_i : Il *logit* per la classe i -esima (l'output grezzo del layer finale).
- K : Il numero totale di classi (es. 10 per CIFAR-10).
- $\exp(\cdot)$: Funzione esponenziale, assicura che tutti i valori siano positivi.
- $\sum_j \exp(z_j)$: Somma degli esponenziali di tutti i logits. Serve a normalizzare i valori in una distribuzione di probabilità.

²Sono i punteggi non normalizzati associati a ciascuna classe, prodotti dall'ultimo layer della rete prima della softmax.

Questa trasformazione assicura che i valori risultanti siano confinati nell'intervallo $[0,1]$ e che la loro somma sia unitaria. Durante l'addestramento, il modello viene ottimizzato minimizzando la cross-entropy loss, una misura che quantifica lo scostamento tra le probabilità predette e la distribuzione reale:

Cross-Entropy Loss:

$$L = - \sum_{i=1}^K y_i \log(\hat{y}_i)$$

- L : Valore della funzione di perdita (loss). Più è basso, meglio è.
- y_i : L'etichetta di verità fondamentale (ground truth) per la classe i , in formato one-hot (1 per la classe corretta, 0 per le altre).
- $\hat{y}_i$: La probabilità predetta dal modello per la classe i , ovvero l'output della funzione softmax.
- $\log$: Logaritmo naturale. Penalizza fortemente le previsioni sbagliate ma con alta confidenza (quando $\hat{y}_i$ è basso per la classe corretta).

Intuitivamente, la cross-entropy penalizza fortemente le predizioni sbagliate con alta confidenza, spingendo la rete a migliorare le sue stime.

2.1.5 Backpropagation e Gradient Descent

L'addestramento di una CNN avviene attraverso backpropagation [11], l'algoritmo che calcola il gradiente, ovvero il vettore delle derivate parziali che indica la direzione di massima crescita della funzione di perdita rispetto a tutti i parametri della rete (pesi e bias). Questo gradiente indica esattamente in quale direzione e di quanto modificare i parametri per ridurre l'errore. Il processo si articola in due fasi:

1. **Forward pass**: l'input viene propagato attraverso la rete, layer dopo layer, fino a produrre l'output finale e calcolare la loss.
2. **Backward pass**: il gradiente della loss viene calcolato a ritroso, partendo dall'output e risalendo verso i primi layer, applicando ripetutamente la chain rule del calcolo differenziale.

Il calcolo dei gradienti permette di determinare la direzione di aggiornamento ottimale per i parametri del modello. L'algoritmo di Stochastic Gradient Descent (SGD) implementa questa regola di aggiornamento nella sua forma essenziale:

Aggiornamento parametri con SGD:

$$\vec{\theta}^{(\text{new})} = \vec{\theta}^{(\text{old})} - \eta \cdot \nabla L(\vec{\theta})$$

- $\vec{\theta}$: Il vettore che contiene tutti i parametri (pesi e bias) del modello.
- η : Il *learning rate*, un iperparametro che controlla l'ampiezza del passo di aggiornamento.
- $\nabla L(\vec{\theta})$: Il gradiente della funzione di perdita rispetto ai parametri $\vec{\theta}$. Indica la direzione di massima crescita della loss.

Nella pratica, vengono spesso utilizzate varianti più sofisticate come Adam [12] o RMSprop, che adattano il learning rate in modo dinamico per ciascun parametro, accelerando la convergenza. Questo ciclo di forward pass, calcolo della loss, backward pass e aggiornamento dei parametri viene ripetuto per molte iterazioni (o epoche, ovvero passaggi completi attraverso l'intero dataset di training) fino a quando la rete non converge a una configurazione di parametri che minimizza la loss sul training set e generalizza bene su dati mai visti.

2.1.6 Activation Functions: non-linearità essenziali

Le reti neurali devono la loro potenza espressiva alla presenza di funzioni di attivazione non lineari. Senza di esse, anche una rete profonda si comporterebbe come un semplice modello lineare, vanificando qualsiasi vantaggio architetturale. La ReLU (Rectified Linear Unit) costituisce la funzione di attivazione più adottata nelle architetture contemporanee. La sua definizione formale è:

Funzione di Attivazione ReLU:

$$\text{ReLU}(x) = \max(0, x)$$

- x : Input al neurone (output della trasformazione lineare precedente).

- $\max(0, x)$: Restituisce x se $x > 0$, altrimenti restituisce 0.

Il comportamento della ReLU è tanto elementare quanto efficace: annulla gli input negativi e preserva inalterati quelli positivi. Oltre ai vantaggi computazionali, questa caratteristica gioca un ruolo cruciale nell'attenuare il fenomeno del *vanishing gradient*, un effetto che nelle reti profonde causa una progressiva attenuazione del segnale di gradiente durante la retropropagazione, compromettendo l'aggiornamento dei parametri negli strati iniziali. Questo problema affliggeva particolarmente le funzioni sigmoidali e tangente iperbolica usate in passato. Il gradiente della ReLU è invece costante (1 per $x > 0$, 0 altrimenti), permettendo un flusso di gradiente stabile durante la backpropagation. Esistono varianti della ReLU, come la **Leaky ReLU** (che invece di azzerare i valori negativi li moltiplica per un piccolo coefficiente, evitando i "neuroni morti") e la **ELU** (Exponential Linear Unit) [13], ma la ReLU standard rimane la scelta predefinita per la maggior parte delle applicazioni [2].

2.2 Architettura del Modello e Scelte di Design

Le CNN moderne non sono monolitiche: esistono decine di architetture diverse, ciascuna con caratteristiche peculiari che le rendono più o meno adatte a specifici compiti e contesti. Comprendere le principali famiglie architetture e le tecniche di design più diffuse è fondamentale per apprezzare le scelte implementative di questo lavoro.

2.2.1 Evoluzione delle architetture CNN

La storia delle CNN è costellata di architetture di riferimento che hanno progressivamente innalzato lo stato dell'arte:

- **AlexNet (2012)** è stata la prima CNN a vincere in modo schiacciante la competizione ImageNet, dimostrando la superiorità del deep learning rispetto ai metodi tradizionali di computer vision [14]. Con 8 layer (5 convoluzionali e 3 fully-connected), ha introdotto l'uso sistematico della ReLU e del dropout.

- **VGG (2014)**, proposta da Simonyan e Zisserman, ha dimostrato che aumentare la profondità della rete usando blocchi ripetuti di convoluzioni 3×3 porta a miglioramenti significativi nelle prestazioni [9]. Le varianti VGG-16 e VGG-19, con rispettivamente 16 e 19 layer, sono diventate rapidamente un punto di riferimento per il transfer learning.
- **ResNet (2015)**, sviluppata da He et al., ha rivoluzionato il campo introducendo le residual connections (o skip connections), che permettono di addestrare reti estremamente profonde (fino a 152 layer o più) senza incorrere nel problema del vanishing gradient [2].
- Altre architetture come **Inception (GoogLeNet)** [15], **DenseNet**, **EfficientNet** e le più recenti **Vision Transformers** [16] hanno continuato a spingere i confini delle prestazioni, introducendo nuove idee come convoluzioni multi-scala, connessioni dense tra layer, o l'uso di meccanismi di attenzione.

2.2.2 Batch Normalization: stabilità e velocità

Per rendere il training più stabile e accelerare la convergenza, è stata introdotta nel 2015 la Batch Normalization (BatchNorm) [17]. Questa tecnica agisce come un "regolatore interno" della rete, normalizzando gli output di ogni layer in modo che mantengano una distribuzione consistente durante l'addestramento. Operativamente, BatchNorm standardizza i dati sottraendo la media e dividendo per la deviazione standard calcolate sul batch corrente:

$$x_{\text{norm}} = \frac{x - \mu_{\text{batch}}}{\sqrt{\sigma_{\text{batch}}^2 + \epsilon}}$$

- x : input del layer
- μ_{batch} : media calcolata sul mini-batch corrente
- σ_{batch}^2 : varianza calcolata sul mini-batch corrente
- ϵ : termine di stabilità numerica (tipicamente 10^{-5})

Dopo la normalizzazione, vengono applicati due parametri apprendibili (γ e β) che permettono alla rete di recuperare la capacità espressiva:

$$y = \gamma \cdot x_{\text{norm}} + \beta$$

- γ : parametro di scala apprendibile
- β : parametro di shift apprendibile
- y : output normalizzato e trasformato

Stabilizza il training riducendo il fenomeno dell'*internal covariate shift*, ovvero la variazione continua della distribuzione degli input a ciascun layer durante l'addestramento. Inoltre, la normalizzazione esplicita consente l'utilizzo di learning rate più elevati, rendendo l'ottimizzazione più efficiente e accelerando la convergenza del modello, come mostrato in Figura 2.2. Esso introduce un effetto regolarizzante dovuto alla stocasticità indotta dal calcolo di media e varianza sui mini-batch, contribuendo a ridurre l'overfitting³.

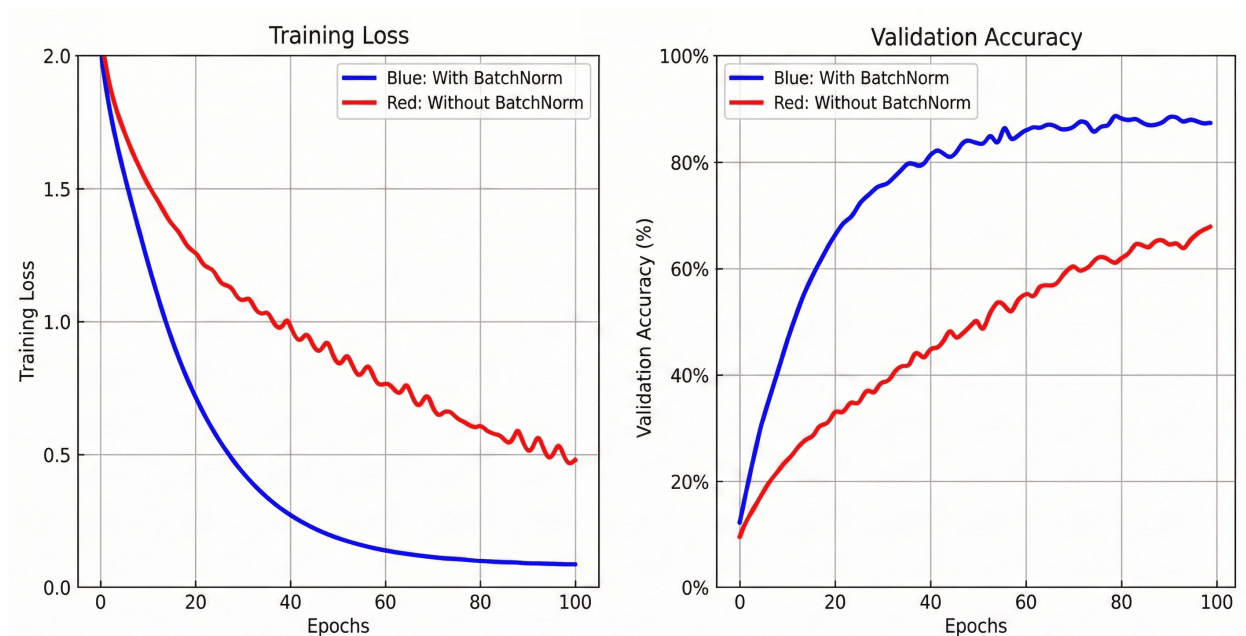


Figura 2.2: Confronto tra una CNN addestrata con e senza Batch Normalization.

Oggi, la Batch Normalization è praticamente onnipresente nelle architetture CNN moderne e rappresenta uno standard de facto.

³Il fenomeno per cui la rete memorizza il training set invece di apprendere pattern generalizzabili.

2.2.3 Dropout: regolarizzazione contro l'overfitting

Un altro strumento cruciale per evitare l'overfitting è il Dropout, proposto da Hinton et al. nel 2012 [18].

$$y_{\text{inference}} = p \cdot y_{\text{training}}$$

- y_{training} : output durante il training (con dropout attivo)
- p : probabilità di retention (tipicamente 0.5)
- $y_{\text{inference}}$: output durante l'inferenza (con tutti i neuroni attivi)

Durante il training, il dropout "spegne" casualmente una frazione (tipicamente 50%) dei neuroni di un layer, impostando il loro output a zero. Questo costringe la rete a non fare affidamento su specifici neuroni, distribuendo l'informazione in modo più robusto e riducendo la co-adattazione tra neuroni. Durante l'inferenza, tutti i neuroni sono attivi, ma i loro output vengono scalati per compensare il fatto che durante il training solo una frazione era attiva. Il dropout è particolarmente efficace nei fully connected layers, che tendono a essere i più soggetti a overfitting a causa dell'elevato numero di parametri.

2.2.4 Adaptive Average Pooling: flessibilità nell'output

Una tecnica più recente e sempre più diffusa è l'Adaptive Average Pooling [19].

A differenza del pooling tradizionale, che ha dimensioni fisse (es. 2×2), l'adaptive pooling produce un output di dimensione desiderata indipendentemente dalla dimensione dell'input. Ad esempio, un Adaptive Average Pooling con output 1×1 calcola la media di tutta la feature map, riducendola a un singolo valore per canale. Questo è particolarmente utile perché:

- Rende l'architettura indipendente dalla dimensione dell'input: la stessa rete può processare immagini di dimensioni diverse.
- Sostituisce elegantemente il flattening⁴ seguito da fully connected layers molto grandi, riducendo il numero di parametri.

⁴Si intende la trasformazione di una feature map multidimensionale in un vettore unidimensionale

- Introduce una forma di contesto globale: ogni valore nell'output tiene conto dell'intera feature map.

2.3 Dataset CIFAR-10

Il dataset CIFAR-10 (Canadian Institute For Advanced Research, 10 classes) è uno dei benchmark⁵ più iconici e utilizzati nel campo della computer vision. Introdotto da Alex Krizhevsky, Vinod Nair e Geoffrey Hinton nel 2009 [3], è diventata la palestra standard per testare nuove architetture, tecniche di regolarizzazione, e, più recentemente, metodi di robustezza avversariale.

2.3.1 Struttura e caratteristiche del dataset

CIFAR-10 consiste di 60.000 immagini a colori (RGB, 3 canali) di dimensione 32×32 pixel, suddivise equamente in 10 classi mutualmente esclusive:

1. **Airplane** (aeroplano)
2. **Automobile** (automobile)
3. **Bird** (uccello)
4. **Cat** (gatto)
5. **Deer** (cervo)
6. **Dog** (cane)
7. **Frog** (rana)
8. **Horse** (cavallo)
9. **Ship** (nave)
10. **Truck** (camion)

⁵In ambito machine learning, è un dataset o una collezione di problemi standardizzata usata per confrontare le prestazioni di diversi modelli o algoritmi.

Il dataset è diviso in 50.000 immagini di training e 10.000 immagini di test, con 6.000 immagini per classe nel training set e 1.000 per classe nel test set. Questa suddivisione è fissa e standard, permettendo confronti diretti tra diversi lavori pubblicati.

2.3.2 Perché CIFAR-10 è un benchmark importante

Nonostante la sua apparente semplicità (immagini piccole, solo 10 classi) CIFAR-10 presenta sfide non banali che lo rendono un test significativo per i modelli di visione artificiale: In Figura 2.3 sono riportati esempi rappresentativi delle dieci classi del dataset, al fine di fornire una visualizzazione intuitiva della varietà semantica del dataset.

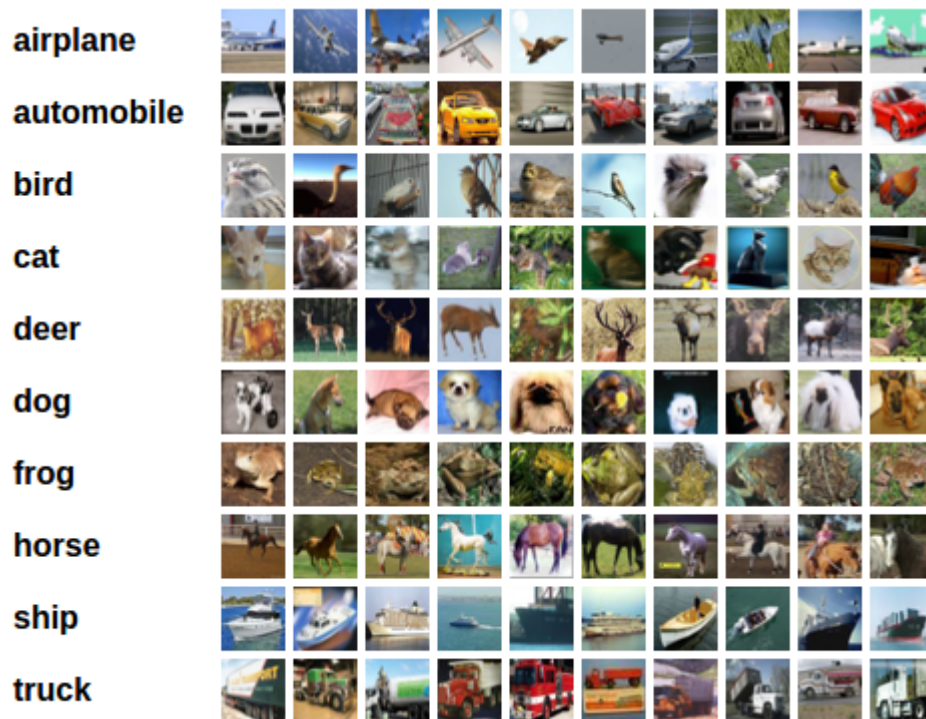


Figura 2.3: Esempi rappresentativi delle dieci classi del dataset CIFAR-10.

- **Bassa risoluzione:** a 32×32 pixel, molti dettagli visivi sono persi o fortemente compressi. Un modello deve imparare a estrarre caratteristiche discriminanti anche da informazione visiva limitata.

- **Variabilità intra-classe:** le immagini all'interno della stessa classe possono essere molto diverse tra loro. Un aereo può essere ripreso da diverse angolazioni, con diversi colori, in volo o a terra, di giorno o al tramonto.
- **Somiglianze inter-classe:** alcune classi sono visivamente simili e facilmente confondibili. Distinguere tra un gatto e un cane, o tra un cervo e un cavallo, richiede al modello di cogliere sottili differenze morfologiche.
- **Background e contesto variabili:** le immagini provengono da fotografie reali, con sfondi complessi e variati. Il modello deve imparare a focalizzarsi sull'oggetto principale ignorando elementi di disturbo.
- **Dimensione gestibile:** con 60.000 immagini, CIFAR-10 è abbastanza grande da permettere l'addestramento di reti profonde senza overfitting eccessivo (con le giuste tecniche di regolarizzazione), ma abbastanza piccolo da rendere gli esperimenti computazionalmente fattibili anche senza hardware specializzato di altissimo livello.

Limitazioni e prospettive

È importante riconoscere che CIFAR-10, per quanto utile, ha delle limitazioni intrinseche:

- Le immagini 32×32 sono molto più semplici rispetto a immagini ad alta risoluzione (es. ImageNet a 224×224 o superiori).
- Le 10 classi sono una semplificazione estrema della complessità del mondo reale.
- Il dataset è relativamente "vecchio" (2009) e le immagini provengono da una distribuzione che potrebbe non riflettere la varietà visiva dei dati moderni.

Tuttavia, proprio queste limitazioni rendono CIFAR-10 uno strumento didattico e di ricerca insostituibile: permette di isolare e studiare fenomeni fondamentali – come la vulnerabilità avversariale – senza essere sopraffatti dalla complessità, e di condurre esperimenti riproducibili in tempi ragionevoli.

2.4 Adversarial Machine Learning: Definizioni e Concetti Fondamentali

Nel 2013 è stato scoperto che, aggiungendo perturbazioni minime e spesso impercettibili a un'immagine, era possibile indurre una rete neurale profonda, anche molto accurata, a classificare l'immagine in modo completamente errato [20]. Questa vulnerabilità non rappresentava un bug isolato di un particolare modello, ma una proprietà intrinseca delle reti neurali profonde. Da quella scoperta, il campo dell'Adversarial Machine Learning è cresciuto rapidamente, mostrando che praticamente tutti i modelli di deep learning (non solo per la visione artificiale, ma anche per il Natural Language Processing, il riconoscimento vocale e altri domini) sono vulnerabili a manipolazioni di questo tipo.

2.4.1 Cos'è un esempio avversariale?

Un esempio avversariale è un input creato intenzionalmente aggiungendo una piccola perturbazione a un esempio legittimo, con l'obiettivo di ingannare un modello di machine learning [20]. Formalmente, dato un input originale $\mathbf{x}$ con etichetta corretta y , un esempio avversariale $\mathbf{x}_{\text{adv}}$ è costruito come:

$$\mathbf{x}_{\text{adv}} = \mathbf{x} + \delta$$

- $\mathbf{x}$: immagine originale di input
- δ : perturbazione avversariale
- $\mathbf{x}_{\text{adv}}$: esempio avversariale risultante

L'obiettivo è scegliere δ tale che:

1. La perturbazione sia "piccola" secondo una metrica appropriata: $\|\delta\|_p \leq \epsilon$
2. Il modello produca una predizione errata: $f(\mathbf{x}_{\text{adv}}) \neq y$

2.4.2 Norme di perturbazione: ℓ_∞ , ℓ_2 , ℓ_1 , ℓ_0

Le perturbazioni avversariali possono essere misurate secondo diverse norme [21], ciascuna con significato e implicazioni diverse.

Norma ℓ_∞ (norma infinito)

$$\|\delta\|_\infty = \max_i |\delta_i|$$

La norma ℓ_∞ misura la massima deviazione su qualsiasi singolo pixel. Limitare $\|\delta\|_\infty \leq \epsilon$ significa che ogni pixel può essere modificato al massimo di $\pm\epsilon$. È la norma più utilizzata in adversarial machine learning poiché:

- ϵ rappresenta direttamente lo spostamento massimo per pixel;
- garantisce che nessun pixel subisca variazioni eccessive;
- consente attacchi efficienti come FGSM e PGD [5].

Norma ℓ_2 (norma euclidea)

$$\|\delta\|_2 = \sqrt{\sum_i \delta_i^2}$$

La norma ℓ_2 misura la distanza euclidea della perturbazione, permettendo variazioni più grandi su alcuni pixel purché l'energia totale rimanga limitata. Gli attacchi ℓ_2 tendono a produrre perturbazioni più diffuse e visivamente naturali.

Norma ℓ_1 (norma Manhattan)

$$\|\delta\|_1 = \sum_i |\delta_i|$$

La norma ℓ_1 produce perturbazioni più sparse, modificando pochi pixel in modo significativo e lasciando la maggior parte invariata.

Norma ℓ_0 (norma zero)

$$\|\delta\|_0 = \text{numero di elementi non nulli in } \delta$$

La norma ℓ_0 conta semplicemente quanti pixel sono stati modificati, indipendentemente dall'entità. È tipica di attacchi localizzati come il “one-pixel attack”.

In questo lavoro ci concentriamo esclusivamente sulla norma ℓ_∞ , seguendo la convenzione dominante nella letteratura su CIFAR-10.

2.4.3 Threat Model: white-box vs black-box

Un aspetto fondamentale nell'analisi degli attacchi avversariali è il threat model, ovvero il grado di conoscenza dell'attaccante sul modello target.

White-box attacks

In uno scenario white-box, l'attaccante dispone di accesso completo al modello [5]: architettura, pesi, funzione di loss e gradienti. Questo rappresenta il caso peggiore in termini di sicurezza, ma è anche il più studiato, in quanto consente di valutare la vulnerabilità intrinseca del modello.

Black-box attacks

Nel caso black-box, l'attaccante può solo interrogare il modello e osservare gli output, senza conoscere architettura, parametri o gradienti [22]. Questi attacchi sono più realistici in contesti applicativi (ad esempio, servizi cloud di ML) ma generalmente meno efficaci. Tuttavia, la proprietà di transferability permette spesso di trasferire attacchi da un modello all'altro.

Gray-box attacks

Scenario intermedio in cui l'attaccante conosce solo alcune informazioni (ad esempio l'architettura ma non i pesi). Nel presente lavoro si adotta un modello white-box, che rappresenta la condizione più sfavorevole per il difensore.

2.4.4 Budget di perturbazione (ϵ)

Il parametro ϵ definisce il budget di perturbazione, cioè l'ampiezza massima consentita della perturbazione avversariale. Valori piccoli di ϵ rendono le perturbazioni impercettibili ma meno efficaci, mentre valori troppo grandi le rendono visibili. Per CIFAR-10 (pixel normalizzati in $[0, 1]$), si adotta comunemente $\epsilon = 0.03 \approx 8/255$ come compromesso standard [5].

2.4.5 Transferability: la vulnerabilità condivisa

Una proprietà cruciale degli esempi avversariali è la transferability: perturbazioni generate per un modello f_A tendono spesso a ingannare anche un modello diverso f_B [22]. Formalmente:

$$\mathbb{P}[f_B(\mathbf{x} + \delta_A) \neq y] \gg \mathbb{P}[f_B(\mathbf{x}) \neq y]$$

- δ_A : perturbazione generata per il modello f_A
- f_B : modello diverso

Questo fenomeno implica che i modelli di deep learning condividono vulnerabilità strutturali comuni, anche con architetture e pesi differenti. Ciò ha importanti implicazioni:

- **Sicurezza:** permette attacchi *black-box* per trasferimento;
- **Ricerca:** indica vulnerabilità sistemiche legate alle rappresentazioni apprese;
- **Difesa:** la diversità architetturale da sola non garantisce robustezza.

Targeted vs Untargeted attacks

Gli attacchi avversariali possono essere classificati in base al loro obiettivo:

- **Untargeted attacks**

$$\min_{\delta} \|\delta\|_p \quad \text{s.t.} \quad f(\mathbf{x} + \delta) \neq y$$

L'obiettivo è indurre il modello a sbagliare la classificazione, senza specificare la classe predetta. Attacchi come FGSM e PGD sono tipicamente di questo tipo.

- **Targeted attacks**

$$\min_{\delta} \|\delta\|_p \quad \text{s.t.} \quad f(\mathbf{x} + \delta) = y_{\text{target}}$$

In questo caso, l'attaccante mira a far classificare l'input come una classe specifica y_{target} . Tali attacchi richiedono perturbazioni più ampie o iterative, ma sono particolarmente pericolosi in contesti sensibili (es. riconoscimento facciale).

2.4.6 Perché esistono gli esempi avversariali?

Nonostante anni di ricerca, non esiste ancora una spiegazione teorica univoca della vulnerabilità avversariale. Sono state proposte varie ipotesi.

1. **Ipotesi della linearità locale**

$$f(\mathbf{x} + \delta) \approx f(\mathbf{x}) + \nabla f(\mathbf{x}) \cdot \delta$$

In spazi ad alta dimensionalità, anche perturbazioni molto piccole in molte direzioni possono cumulativamente alterare l'output del modello [4].

2. **Ipotesi della boundary**

I confini di decisione appresi dalle reti neurali si trovano spesso molto vicini ai dati naturali, rendendo facile attraversarli con perturbazioni minime.

3. **Ipotesi della dimensionalità**

Lo spazio degli input è estremamente ampio, mentre i dati di training ne occupano solo una frazione minima. Gli esempi avversariali esplorano regioni "vuote" di questo spazio dove il comportamento del modello è meno prevedibile.

4. **Ipotesi delle feature non robuste**

$$\mathbb{E}_{(\mathbf{x}, y)}[\ell(\theta, \mathbf{x}, y)] = \mathbb{E}[\ell_{\text{robusto}}] + \mathbb{E}[\ell_{\text{non-robusto}}]$$

I modelli apprendono pattern altamente predittivi ma fragili rispetto a piccole perturbazioni [23]. Queste feature non robuste spiegano perché un modello possa ottenere alte accuratèzze sui dati naturali ma fallire di fronte a perturbazioni impercettibili.

Nel complesso, è probabile che la vulnerabilità avversariale derivi da una combinazione di questi fattori, rendendola una caratteristica intrinseca dei moderni modelli di deep learning.

2.5 Attacchi Adversariali

Dopo aver stabilito i concetti fondamentali, possiamo ora analizzare in dettaglio i principali metodi di attacco avversariale. In questa sezione ci concentreremo su FGSM e PGD, i due attacchi implementati in questo lavoro, fornendo anche una breve panoramica di altri metodi rilevanti per completezza.

2.5.1 Fast Gradient Sign Method (FGSM)

Proposto da Goodfellow et al. nel 2015 [4], il Fast Gradient Sign Method rappresenta il primo attacco avversariale ampiamente studiato, rimasto popolare per la sua elegante semplicità ed efficienza computazionale.

Intuizione

L'idea fondamentale è muoversi nella direzione del gradiente della funzione di perdita rispetto all'input. FGSM semplifica ulteriormente il processo considerando solo il segno del gradiente, muovendosi di un passo fisso ϵ in quella direzione. Questo rende l'attacco estremamente veloce, richiede solo una forward pass e una backward pass, e garantisce che la perturbazione rispetti esattamente il vincolo ℓ_∞ .

$$\mathbf{x}_{\text{adv}} = \mathbf{x} + \epsilon \cdot \text{sign}(\nabla_{\mathbf{x}} \mathcal{L}(\theta, \mathbf{x}, y))$$

- $\nabla_{\mathbf{x}} \mathcal{L}$: gradiente della loss rispetto all'input
- $\text{sign}(\cdot)$: funzione segno che preserva la direzione ma non la magnitudine

- ϵ : budget di perturbazione (tipicamente 0.03 per CIFAR-10)

Segue l'operazione di clipping per mantenere valori pixel validi:

$$\mathbf{x}_{\text{adv}} = \text{clip}(\mathbf{x}_{\text{adv}}, 0, 1)$$

Per chiarire geometricamente la differenza tra un attacco single-step come FGSM e un attacco iterativo come PGD, la Figura 2.4 mostra una rappresentazione intuitiva del loss landscape e delle rispettive traiettorie di attacco sotto vincolo ℓ_∞ .

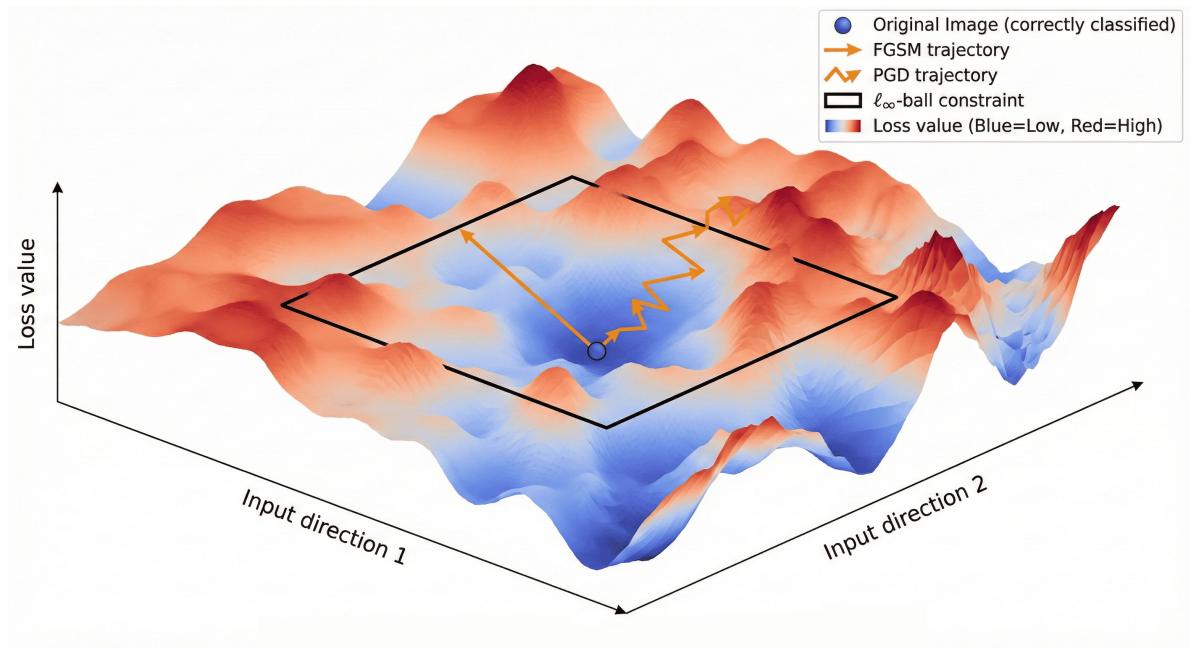


Figura 2.4: Proiezione 2D del loss landscape in uno spazio di input ad alta dimensionalità.

Vantaggi DI FGSM

- **Velocità:** richiede solo una forward e una backward pass, rendendolo computazionalmente economico.
- **Semplicità:** facile da implementare e comprendere
- **Garanzia sul budget:** la perturbazione è sempre esattamente $\|\delta\|_\infty = \epsilon$
- **Baseline efficace:** nonostante la semplicità, FGSM è sorprendentemente efficace contro modelli non robusti

Limitazioni DI FGSM

- **Single-step:** essendo un attacco one-shot, può rimanere "intrappolato" in minimi locali della loss landscape
- **Direzione sub-ottimale:** prendere solo il segno del gradiente scarta informazione sulla magnitudine, potenzialmente perdendo direzioni più efficaci
- **Meno efficace contro modelli robusti:** modelli addestrati con adversarial training possono resistere relativamente bene a FGSM

Proprio queste limitazioni hanno motivato lo sviluppo di attacchi iterativi più sofisticati, come PGD.

2.5.2 Projected Gradient Descent (PGD)

Introdotta da Madry et al. nel 2018 [5], il Projected Gradient Descent rappresenta uno degli attacchi più forti e più utilizzati per valutare la robustezza avversariale. PGD può essere visto come una generalizzazione iterativa di FGSM: invece di fare un singolo grande passo nella direzione del segno del gradiente, PGD fa molti passi più piccoli, ricalcolando il gradiente ad ogni iterazione e "proiettando" la perturbazione all'interno della ℓ_∞ -ball ammissibile.

$$\mathbf{x}^{t+1} = \Pi_{\mathbf{x}+\mathcal{S}} (\mathbf{x}^t + \alpha \cdot \text{sign}(\nabla_{\mathbf{x}} \mathcal{L}(\theta, \mathbf{x}^t, y)))$$

- α : step size (tipicamente $\alpha = 2\epsilon/T$)
- $\Pi_{\mathbf{x}+\mathcal{S}}$: proiezione sulla ℓ_∞ -ball di raggio ϵ
- T : numero di iterazioni (tipicamente 7 - 40 per CIFAR-10)

La proiezione viene implementata come:

$$\Pi_{\mathbf{x}+\mathcal{S}}(\mathbf{x}') = \text{clip}(\mathbf{x}', \mathbf{x} - \epsilon, \mathbf{x} + \epsilon)$$

Inizializzazione

Un aspetto importante di PGD è l'inizializzazione. Le due strategie più comuni sono:

1. **Inizializzazione dall'immagine originale:** $\mathbf{x}^0 = \mathbf{x}$

Pro: deterministico, riproducibile

Contro: può convergere a minimi locali sub-ottimali

2. **Inizializzazione random:** $\mathbf{x}^0 = \mathbf{x} + \mathcal{U}(-\epsilon, \epsilon)$

Pro: maggiore esplorazione dello spazio, meno probabilità di rimanere intrappolati in minimi locali

Contro: stocastico, risultati possono variare tra esecuzioni.

Nella letteratura, l'inizializzazione random è generalmente preferita e considerata la configurazione standard.

Tabella 2.1: Confronto tra FGSM e PGD rispetto a vari aspetti

Aspetto	FGSM	PGD
Velocità	Molto veloce (1 iterazione)	Più lento (7–100 iterazioni)
Efficacia	Buona su modelli non robusti	Eccellente, stato dell'arte
Uso principale	Adversarial training economico	Valutazione robustezza, adversarial training forte
Garanzia di robustezza	Debole	Forte

2.5.3 Altri Attacchi Rilevanti (cenni)

Per completezza, menzioniamo brevemente altri attacchi adversariali rilevanti, sebbene non siano stati implementati in questo lavoro:

C&W Attack

L'attacco di Carlini e Wagner è basato su ottimizzazione numerica piuttosto che su gradienti first-order. Formula il problema di generare esempi adversariali come un problema di ottimizzazione vincolata [21]:

$$\min_{\delta} \|\delta\|_p + c \cdot f(\mathbf{x} + \delta)$$

dove $f(\cdot)$ è una funzione obiettivo che penalizza classificazioni corrette.

DeepFool

DeepFool DeepFool cerca di trovare la perturbazione minima necessaria per attraversare il confine di decisione. Modellizza localmente il confine come un iperpiano e si muove perpendicolarmente ad esso, iterativamente, fino a causare misclassificazione [24].

One-Pixel Attacks

Un attacco sorprendente che modifica solo un singolo pixel dell'immagine, scegliendo la sua posizione e colore attraverso algoritmi evolutivi [25]. Dimostra che talvolta anche modifiche estremamente localizzate possono ingannare reti neurali, sebbene il tasso di successo sia inferiore rispetto ad attacchi che modificano molti pixel.

Universal Adversarial Perturbations

Invece di generare perturbazioni specifiche per ogni immagine, si cerca una singola perturbazione "universale" che, quando aggiunta a qualsiasi immagine del dataset, causa misclassificazione [26]. L'esistenza di tali perturbazioni rivela ulteriori fragilità strutturali dei modelli di deep learning.

Adversarial Patch

Un attacco fisicamente realizzabile dove si crea un adesivo (patch) che, quando applicato in qualsiasi posizione di un'immagine, causa misclassificazione [27]. Questo

ha importanti implicazioni per la sicurezza nel mondo reale: qualcuno potrebbe stampare un adesivo e attaccarlo su un segnale stradale per ingannare un sistema di guida autonoma.

2.6 Tecniche di Difesa

Di fronte alla minaccia concreta rappresentata dagli attacchi avversariali, la comunità di ricerca ha risposto con una domanda cruciale: come costruire sistemi di machine learning in grado di resistere a queste manipolazioni? Il cammino verso difese efficaci si è rivelato più complesso del previsto, caratterizzato da un continuo dialogo tra innovazioni difensive e contromisure offensive.

Negli ultimi anni abbiamo assistito a un fiorire di tecniche difensive, ciascuna con promesse e limiti diversi. Tuttavia, molte soluzioni inizialmente promettenti hanno mostrato il loro volto più fragile quando esposte ad attacchi adattativi - versioni perfezionate degli attacchi tradizionali, specificamente progettate per sondare i punti deboli di ogni nuova difesa.

Questo ciclo di scoperte e disillusioni ha insegnato alla comunità una lezione fondamentale: l'efficacia di una difesa non va misurata contro attacchi standard, ma contro avversari determinati che studiano e adattano le loro strategie. Proprio da questo processo di affinamento metodologico è emerso con sempre maggiore chiarezza che l'*adversarial training* rappresenta, allo stato attuale, uno degli approcci difensivi più solido e affidabile.

2.6.1 Adversarial Training

L'*adversarial training* è concettualmente semplice, ma potente: invece di addestrare il modello solo su dati puliti, lo si addestra su un mix di esempi originali ed esempi avversariali generati durante il training stesso. L'intuizione è che, esponendo il modello a perturbazioni avversariali durante l'apprendimento, esso impari rappresentazioni più robuste e confini di decisione più stabili.

Formulazione min-max

Goodfellow et al. (2015) hanno proposto la prima versione di adversarial training usando FGSM [4]. Madry et al. (2018) l'hanno poi formalizzato come un problema di ottimizzazione min-max [5].:

$$\min_{\theta} \mathbb{E}_{(x,y) \sim \mathcal{D}} \left[\max_{\delta \in \mathcal{S}} \mathcal{L}(\theta, x + \delta, y) \right]$$

- θ : parametri del modello da ottimizzare
- $\mathcal{D}$: distribuzione dei dati di training
- $\mathcal{S}$: spazio delle perturbazioni ammissibili ($\|\delta\|_{\infty} \leq \epsilon$)
- $\mathcal{L}$: funzione di perdita del modello
- max: ricerca della perturbazione più dannosa (attaccante)
- min: ottimizzazione dei parametri robusti (difensore)

Questa formulazione cattura elegantemente l'idea di adversarial training.

Perché funziona?

L'adversarial training è efficace perché:

- **Regolarizza lo spazio delle rappresentazioni:** costringe il modello a imparare features più robuste e meno sensibili a piccole perturbazioni
- **Espande i confini di decisione:** le regioni di classificazione corretta diventano più ampie
- **Riduce la curvatura locale:** la funzione appresa diventa meno sensibile ai gradienti

FGSM training vs PGD training

Due varianti principali di adversarial training si distinguono per l'attacco utilizzato per generare esempi durante il training:

1. **FGSM training:** più veloce ed economico, ma conferisce robustezza limitata principalmente contro FGSM stesso
2. **PGD training:** più costoso computazionalmente ma fornisce robustezza generale e forte, considerato stato dell'arte

Come osservato in vari studi, la robustezza trasferisce verso il basso ma non verso l'alto: un modello addestrato con PGD è robusto anche contro FGSM, ma non viceversa [5].

Limitazioni: Nonostante sia la difesa più efficace, l'adversarial training presenta limiti significativi:

- **Costo computazionale elevato:** può richiedere 7-10 volte più tempo del training standard
- **Trade-off accuratezza-robustezza:** riduce inevitabilmente l'accuratezza sui dati puliti
- **Robustezza limitata al budget ϵ :** protegge solo contro perturbazioni fino a una certa dimensione
- **Richiede più epoche:** la convergenza è più lenta rispetto al training standard

2.6.2 Altre Difese

Oltre all'adversarial training, sono state proposte numerose strategie alternative. Forniremo qui una panoramica critica delle principali famiglie di difese, evidenziando i loro limiti.

Defensive Distillation

Proposta da Papernot et al. (2016) [28], adatta una tecnica di compressione dei modelli per aumentare la robustezza. Il modello viene addestrato su "soft labels" (distribuzioni di probabilità regolarizzate) invece che su "hard labels", con l'idea di ridurre la sensibilità del gradiente.

Input Transformations

Questa famiglia di difese applica trasformazioni all'input prima di passarlo alla rete, con l'obiettivo di "ripulire" eventuali perturbazioni avversariali. Tecniche comuni includono:

- **JPEG compression:** comprimere l'immagine per rimuovere artefatti ad alta frequenza.
- **Bit-depth reduction:** ridurre il numero di bit per pixel
- **Image quilting:** ricostruire l'immagine usando patch di un database
- **Total variation minimization:** smoothing attraverso ottimizzazione

Adversarial Detection

Invece di rendere il modello robusto, queste tecniche cercano di rilevare esempi avversariali e rifiutarli. Approcci includono:

- **Statistical tests:** analizzare le statistiche degli activation layers.
- **Auxiliary networks:** addestrare una rete separata per distinguere esempi clean da adversarial.
- **Confidence-based detection:** sfruttare il fatto che esempi avversariali spesso producono distribuzioni di output anomale.

Certified Defenses

Una direzione di ricerca promettente ma ancora limitata nella pratica sono le certified defenses, che forniscono garanzie matematiche sulla robustezza. Tecniche principali:

- **Randomized smoothing:** aggiunge rumore gaussiano all'input e "vota" tra predizioni multiple, fornendo un certificato probabilistico di robustezza [29]
- **Interval bound propagation:** calcola bounds formali sull'output per ogni possibile perturbazione nell'input

- **Lipschitz constraints:** limita la costante di Lipschitz della rete per garantire smooth output

Ensemble Methods e Diversity

L'idea è di addestrare multipli modelli con architetture o inizializzazioni diverse, sperando che esempi avversariali non trasferiscano efficacemente tra modelli.

Input Preprocessing Robustness

Tecniche come **Feature Squeezing** o **PixelDefend** cercano di "normalizzare" gli input per rimuovere perturbazioni.

Tabella 2.2: Confronto delle principali tecniche di difesa adversarial

Difesa	Vantaggi	Svantaggi	Stato
Adversarial Training	Robustezza empirica forte	Trade-off accuratezza	Raccomandata
Defensive Distillation	Facile implementazione	Vulnerabile a C&W	Superata
Input Transformations	Basso costo computazionale	Aggirabile con gradienti	Limitata
Certified Defenses	Garanzie matematiche	ϵ piccolo, costo alto	Sperimentale

In questo lavoro, ci concentreremo sull'adversarial training come tecnica di riferimento, implementando sia la variante FGSM che quella PGD per confrontarne l'efficacia.

2.7 Il Trade-off Accuratezza-Robustezza

Una delle scoperte più importanti e, per certi versi, più frustranti dell'adversarial machine learning è l'esistenza di un trade-off fondamentale tra accuratezza standard e robustezza avversariale. In termini semplici: un modello addestrato per essere robusto contro attacchi avversariali tende sistematicamente a perdere accuratezza sui

dati puliti, e viceversa. Questo non è un semplice artefatto di tecniche di difesa imperfette, ma sembra riflettere una tensione intrinseca negli obiettivi di apprendimento. Comprendere questo trade-off è cruciale per sviluppare sistemi di ML realistici e per prendere decisioni informate su quanto sacrificare un obiettivo per l'altro.

Evidenze empiriche

Il trade-off è stato documentato in modo estensivo nella letteratura. I lavori seminali di Madry et al. (2018) [5] e Tsipras et al. (2019) [30] hanno mostrato che:

- Modelli adversarially trained su CIFAR-10 tipicamente perdono 7-15 punti percentuali di accuratezza standard rispetto ai modelli clean
- Su ImageNet, il gap può essere ancora più pronunciato (10-20 punti percentuali)
- Questo pattern si ripete attraverso diverse architetture, dataset, e tecniche di adversarial training

Perché esiste questo trade-off?

Diverse teorie sono state proposte per spiegare l'origine del trade-off:

1. Features robuste vs features non robuste

Ilyas e collaboratori hanno proposto una teoria influente [23]: i dataset naturali contengono due tipi di features:

- **Features robuste:** pattern che sono semanticamente significativi e stabili rispetto a piccole perturbazioni (es. la forma generale di un oggetto)
- **Features non robuste:** pattern che sono predittivi ma fragili (es. texture ad alta frequenza, artefatti di compressione, correlazioni spurie)

I modelli standard imparano a sfruttare entrambi i tipi di features perché questo massimizza l'accuratezza sul training set. Le features non robuste sono spesso altamente predittive e permettono di ottenere performance migliori. L'adversarial training, invece, forza il modello a ignorare le features non robuste (perché sono facilmente manipolabili) e a concentrarsi solo su quelle robuste. Questo

limita la capacità del modello di sfruttare tutta l'informazione disponibile nei dati, riducendo l'accuratezza standard.

2. **Capacità del modello limitata** Zhang et al. (2019) hanno argomentato [31] che il trade-off potrebbe essere in parte dovuto alla capacità limitata delle reti neurali. Imparare rappresentazioni robuste richiede maggiore complessità e capacità espressiva. Con risorse computazionali e architetture limitate, il modello deve "scegliere" se specializzarsi su dati puliti o su dati avversariali. Esperimenti con reti molto più grandi (es. Wide ResNet con 10x più parametri) mostrano un trade-off ridotto ma non eliminato, suggerendo che la capacità è un fattore ma non l'unico.
3. **Complessità intrinseca del problema** Alcuni ricercatori sostengono che classificare correttamente in presenza di perturbazioni avversariali sia intrinsecamente un problema più difficile rispetto alla classificazione standard. Il modello deve non solo riconoscere l'oggetto, ma farlo in modo sufficientemente robusto da resistere a perturbazioni ottimizzate per ingannarlo, un compito fondamentalmente più complesso.
4. **Conflitto tra obiettivi di generalizzazione** La generalizzazione standard (da training set a test set) e la generalizzazione robusta (da esempi clean a esempi avversariali) potrebbero richiedere bias induttivi diversi e potenzialmente incompatibili. Ottimizzare per uno può danneggiare l'altro.

2.7.1 Quantificare il trade-off

È possibile visualizzare e quantificare il trade-off attraverso diverse metriche:

Curva accuratezza-robustezza

Si può addestrare una famiglia di modelli con diversi livelli di adversarial training (variando ϵ o il rapporto clean/adversarial) e tracciare l'accuratezza standard vs robusta. Tipicamente si ottiene una curva decrescente: aumentare la robustezza costa accuratezza.

Pareto frontier

Il concetto di Pareto efficiency può essere applicato: un modello è Pareto-efficiente se non è possibile migliorare né l'accuratezza standard né la robustezza senza peggiorare l'altra. La frontiera di Pareto rappresenta il "miglior trade-off possibile" data una certa architettura e dataset.

Robust overfitting

Un fenomeno interessante osservato da Rice et al. (2020) [32] è il robust overfitting: durante l'adversarial training, la robust accuracy sul training set continua a migliorare, ma quella sul test set peggiora dopo un certo numero di epoche. Questo suggerisce che il modello "memorizza" gli esempi adversariali del training set senza generalizzare la robustezza. Questo fenomeno richiede early stopping attento o tecniche di regolarizzazione specifiche.

2.7.2 Implicazioni pratiche

Il trade-off accuratezza-robustezza ha profonde implicazioni per l'applicazione pratica del machine learning:

1. **Contesto applicativo determina la scelta** Non esiste una soluzione omni-comprendensiva. La decisione su quanto investire in robustezza dipende dal contesto:

Tabella 2.3: Trade-off accettabili in diversi contesti applicativi

Applicazione	Priorità	Accettabile	Inaccettabile
Guida autonoma	Robustezza	-5% accuracy	Falsi negativi
Diagnostica medica	Robustezza	-8% accuracy	Errori critici
Raccomandazioni	Accuratezza	Vulnerabilità minore	-15% accuracy
Sicurezza	Bilanciato	-10% accuracy	False acceptance

- **Applicazioni critiche per la sicurezza** (guida autonoma, sistemi medici, riconoscimento facciale per accessi): priorità alla robustezza, anche a costo di accuratezza
 - **Applicazioni consumer non critiche** (raccomandazioni, filtri fotografici): priorità all'accuratezza, la robustezza è secondaria
 - **Sistemi finanziari/fraud detection**: bilanciamento attento, poiché sia falsi positivi che falsi negativi hanno costi
2. **Certificazione e garanzie** Per applicazioni critiche, potrebbe essere necessario fornire garanzie formali sulla robustezza, anche se questo costa significativamente in termini di accuratezza. Le certified defenses, nonostante le loro limitazioni, potrebbero essere l'unica scelta accettabile in certi contesti regolamentati.
 3. **Design di architetture specifiche** La ricerca si sta orientando verso architetture progettate specificamente per robustezza, che potrebbero ridurre il trade-off. Alcuni esempi includono le reti con "feature purification" incorporata, le architetture con meccanismi di attenzione robusti e i design ispirati alla neuroscienza che mimano caratteristiche del sistema visivo umano.
 4. **Ensemble e sistemi ibridi** In alcuni scenari, potrebbe essere sensato utilizzare sistemi ibridi: un modello accurato per dati normali e un modello robusto attivato solo quando si rileva potenziale attacco. Questo richiede però detection affidabile, che come visto è difficile.

Il trade-off è inevitabile?

Una domanda aperta è se il trade-off sia fondamentale e inevitabile o se sia possibile, con le giuste tecniche, avere sia accuratezza che robustezza. Le evidenze attuali suggeriscono che un certo grado di trade-off sia intrinseco, ma la sua entità può essere ridotta:

Architetture più grandi e capienti riducono il gap. Tecniche di training più sofisticate (TRADES, MART, AWP) migliorano entrambi gli obiettivi. Pretraining su dataset più grandi può fornire rappresentazioni che generalizzano meglio.

Tuttavia, nessuna tecnica attuale elimina completamente il trade-off. Comprendere se ciò sia possibile rimane una domanda aperta e fondamentale nel campo.

2.8 Metriche di Valutazione

Per valutare correttamente l'efficacia degli attacchi adversariali e delle difese, è necessario un insieme robusto di metriche che catturino diversi aspetti delle prestazioni del modello. In questa sezione presentiamo le principali metriche utilizzate sia per la valutazione standard che per quella adversariale [4, 5].

2.8.1 Metriche Standard

Le metriche standard per problemi di classificazione forniscono la baseline per valutare le prestazioni sui dati puliti.

Accuracy (Accuratezza)

La metrica più semplice e intuitiva, definita come la frazione di predizioni corrette:

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}}$$

$$\text{Accuracy}_{\text{multi-classe}} = \frac{1}{N} \sum_{i=1}^N \mathbb{1}(\hat{y}_i = y_i)$$

- **TP (True Positives)**: esempi positivi classificati correttamente come positivi
- **TN (True Negatives)**: esempi negativi classificati correttamente come negativi
- **FP (False Positives)**: esempi negativi erroneamente classificati come positivi
- **FN (False Negatives)**: esempi positivi erroneamente classificati come negativi

Vantaggi: semplice, intuitiva, facile da interpretare.

Limitazioni: può essere fuorviante con dataset sbilanciati (poco rilevante per CIFAR-10 che è bilanciato)

Precision (Precisione)

La frazione di predizioni positive che sono effettivamente corrette:

$$\text{Precision} = \frac{TP}{TP + FP}$$

La precision risponde alla domanda: "Quando il modello dice 'classe A', quanto spesso ha ragione?"

Recall (Richiamo o Sensibilità)

La frazione di esempi positivi che sono stati correttamente identificati:

$$\text{Recall} = \frac{TP}{TP + FN}$$

Il recall risponde alla domanda: "Di tutti gli esempi della 'classe A', quanti ne abbiamo catturati?"

F1-Score

La media armonica di precision e recall, utile quando si vuole un singolo numero che bilanci entrambe:

$$F1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

Per problemi multi-classe, precision, recall e F1 possono essere calcolati in diversi modi:

- **Macro-average:** calcola la metrica per ogni classe e fa la media (tratta tutte le classi ugualmente)
- **Micro-average:** aggrega i contributi di tutte le classi (equivale all'accuracy per dataset bilanciati)
- **Weighted-average:** media pesata per il numero di esempi di ogni classe

Loss (Perdita)

Oltre all'accuracy, è importante monitorare la loss (tipicamente cross-entropy per classificazione):

$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^N \sum_{c=1}^C y_{i,c} \log(p_{i,c})$$

La loss fornisce informazioni più granulari rispetto all'accuracy: anche quando due modelli hanno stessa accuracy, quello con loss più bassa è generalmente preferibile perché le sue predizioni sono più "confidenti" quando corrette.

Confusion Matrix

La confusion matrix è uno strumento visuale potente per capire quali errori sta facendo il modello. Per un problema a C classi, è una matrice C×C. Ella permette di identificare:

- **Classi problematiche:** classi con bassa accuracy (righe con molti errori)
- **Confusioni sistematiche:** coppie di classi frequentemente confuse (es. gatto-cane, cervo-cavallo)
- **Pattern di errore:** se gli errori sono distribuiti uniformemente o concentrati su poche classi

2.8.2 Metriche per Adversarial Robustness

Quando si valutano modelli sotto attacco avversariale, sono necessarie metriche aggiuntive.

Robust Accuracy

L'accuracy calcolata su esempi adversariali invece che su dati puliti:

$$\text{Robust Accuracy} = \frac{1}{N} \sum_{i=1}^N \mathbb{1}(f(x_i + \delta_i) = y_i)$$

Questa è la metrica più importante per valutare la robustezza. Un modello con alta robust accuracy è effettivamente resistente agli attacchi.

Attack Success Rate (ASR)

La frazione di esempi originariamente classificati correttamente che vengono misclassificati dopo l'attacco:

$$\text{ASR} = \frac{|\{i : f(x_i) = y_i \wedge f(x_i + \delta_i) \neq y_i\}|}{|\{i : f(x_i) = y_i\}|}$$

ASR e robust accuracy sono complementari: ASR alta = robust accuracy bassa.

Clean Accuracy vs Robust Accuracy

È essenziale riportare entrambe le metriche per caratterizzare completamente un modello:

- **Clean accuracy:** prestazioni su dati non perturbati
- **Robust accuracy (FGSM):** prestazioni sotto attacco FGSM con ϵ specificato
- **Robust accuracy (PGD):** prestazioni sotto attacco PGD con ϵ , num_iter specificati

Robustness Gap

La differenza tra clean accuracy e robust accuracy:

$$\text{Gap} = \text{Clean Accuracy} - \text{Robust Accuracy}$$

Il gap quantifica quanto il modello "perde" quando attaccato. Gap piccolo = buona robustezza relativa.

CAPITOLO 3

Metodo di Ricerca

Questo capitolo descrive il framework metodologico sviluppato per investigare la robustezza delle reti neurali agli attacchi avversariali. Attraverso un approccio sperimentale sistematico e riproducibile, vengono delineate le fasi di progettazione, implementazione, addestramento e valutazione comparativa di modelli con diverse strategie di difesa. Tutte le scelte tecniche qui descritte sono state guidate dalle domande di ricerca presentate nella Sezione 1.3, garantendo che ogni componente dello studio sia direttamente finalizzato a raccogliere evidenze empiriche pertinenti. L'implementazione completa del codice sorgente è disponibile nel repository GitHub del progetto <https://github.com/RVigliotta/cifar10-adversarial-attacks>.

3.1 Domande di Ricerca e Design Sperimentale

Il design sperimentale di questo lavoro è stato strutturato per rispondere sistematicamente alle cinque domande di ricerca enunciate nell'Introduzione. Ogni scelta metodologica è direttamente motivata dalla necessità di raccogliere evidenze empiriche per una o più RQ.

RQ1: Vulnerabilità delle CNN standard

Questa RQ guida la necessità di avere un modello di riferimento "non robusto" contro cui confrontare le difese. È stato addestrato un modello baseline esclusivamente su dati naturali (clean). La sua vulnerabilità è stata quantificata misurando il crollo dell'accuratezza quando sottoposto agli attacchi FGSM e PGD, utilizzando metriche come l'Attack Success Rate.

RQ2: Efficacia dell'adversarial training

Sono stati addestrati due modelli difensivi tramite adversarial training, uno usando l'attacco FGSM e l'altro l'attacco PGD. La loro efficacia è stata valutata confrontando la loro robustezza (accuratezza sotto attacco) con quella del modello baseline.

RQ3: Trade-off accuratezza-robustezza

Tutti i modelli sono stati valutati sia su dati puliti che adversariali. Il trade-off è stato analizzato calcolando la differenza (gap) tra l'accuratezza clean e quella robusta, e visualizzato attraverso grafici comparativi. Questo design permette di rispondere non solo se esiste un trade-off, ma "quanto grande" è e se può essere bilanciato.

RQ4: PGD training vs FGSM training

I due modelli addestrati in modo adversarial (FGSM e PGD) sono stati valutati incrociatamente contro entrambi i tipi di attacco. Questo design permette di verificare se la robustezza ottenuta con un attacco forte (PGD) generalizza meglio verso attacchi più deboli.

RQ5: Costo computazionale

Per ogni fase di addestramento e valutazione sono stati registrati i tempi di esecuzione e le risorse hardware utilizzate (GPU), permettendo di quantificare l'overhead introdotto dall'adversarial training. La scelta di utilizzare Google Colab con GPU T4 (invece di solo CPU) è motivata proprio dalla necessità di gestire l'overhead computazionale dell'adversarial training in tempi ragionevoli.

Considerazioni sul Design

Questo design sperimentale presenta diversi punti di forza:

- **Comparabilità diretta:** Tutti i modelli condividono la stessa architettura, lo stesso dataset, e configurazioni di training omogenee, garantendo che le differenze osservate siano attribuibili all'adversarial training e non ad altri fattori confondenti.
- **Copertura completa:** La combinazione di 3 modelli $\times$ 3 condizioni di valutazione (clean, FGSM, PGD) produce una matrice 3×3 di risultati che permette analisi approfondite e risposte a tutte le RQ.
- **Riproducibilità:** L'uso di seed fissi, checkpointing sistematico, e documentazione dettagliata dei parametri permette la replica completa degli esperimenti.
- **Efficienza:** Sebbene l'adversarial training sia costoso, la scelta di un'architettura compatta (590K parametri) e l'uso di GPU permettono di completare gli esperimenti in tempi gestibili.

Nelle sezioni successive, ogni componente metodologica viene descritta in dettaglio, fornendo le informazioni necessarie per comprendere e replicare il lavoro svolto.

3.2 Panoramica della Metodologia

La metodologia è stata implementata come un flusso di lavoro sequenziale e integrato, illustrato schematicamente nella Figura 3.1. Il processo inizia con la definizione degli obiettivi (RQ) e la configurazione dell'ambiente sperimentale. Segue la preparazione del dataset CIFAR-10 e la definizione dell'architettura di rete TestCNNv2. Il cuore dello studio risiede nel ciclo di addestramento parallelo delle tre varianti del modello (clean, FGSM-trained, PGD-trained), ciascuna con la sua specifica strategia. I modelli addestrati vengono quindi sottoposti a una valutazione multidimensionale che misura le prestazioni in tre condizioni (clean, sotto attacco FGSM, sotto attacco PGD). Infine, i risultati vengono aggregati, visualizzati e analizzati per trarre

conclusioni in risposta a ciascuna RQ. L'intero processo è sorretto da rigorosi protocolli di riproducibilità, tra cui il fissaggio dei seed, il checkpointing sistematico e la documentazione completa di tutti i parametri.

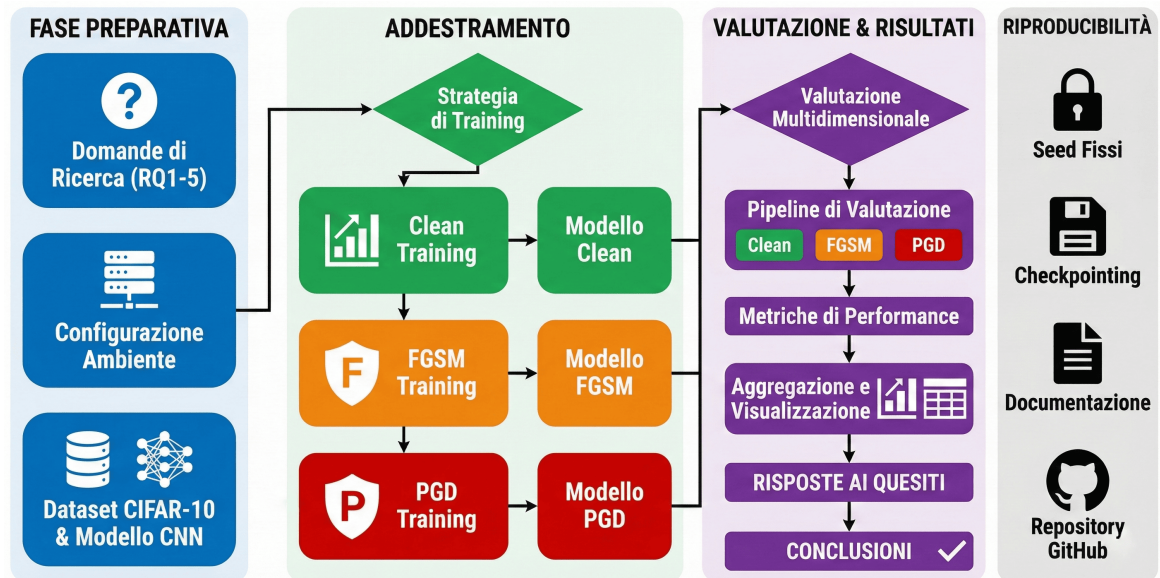


Figura 3.1: Panoramica del progetto

3.3 Setup Sperimentale

Una descrizione trasparente dell'ambiente tecnico costituisce un prerequisito fondamentale per la riproducibilità della ricerca. Per far fronte alle diverse esigenze computazionali del ciclo di sviluppo, è stata impiegata un'infrastruttura ibrida che combina la flessibilità di un ambiente locale con la potenza di calcolo del cloud.

3.3.1 Hardware e Ambiente di Calcolo

Ambiente Locale

È stato utilizzato per le fasi di sviluppo iniziale, debug del codice, pre-processing dei dati, analisi post-hoc dei risultati e generazione delle visualizzazioni. Il sistema era un notebook equipaggiato con processore AMD Ryzen 7 4700U e 16 GB di RAM, con sistema operativo Windows 11, offrendo un'iterazione rapida e un controllo diretto.

Ambiente Cloud (Google Colab Pro)

Tutte le sessioni intensive di training dei modelli e di generazione degli attacchi avversariali sono state condotte in cloud, sfruttando l’accelerazione hardware dedicata di una GPU NVIDIA Tesla T4 con 16 GB di VRAM GDDR6. La scelta è stata dettata dall’onere computazionale dell’adversarial training iterativo (in particolare con PGD), che risulterebbe impraticabile su CPU in tempi ragionevoli. L’utilizzo della GPU ha permesso di ridurre i tempi di training da diverse ore a poche ore per esperimento.

3.3.2 Librerie e Dipendenze Software

L’intero progetto è basato sull’ecosistema **PyTorch**, selezionato per il suo paradigma di programmazione imperativo, il supporto nativo al calcolo differenziale su GPU e la flessibilità nella definizione di modelli e pipeline di training. La Tabella 3.1 elenca le principali librerie utilizzate e il loro scopo specifico.

Tabella 3.1: Panoramica delle principali dipendenze software e delle loro versioni.

Libreria	Versione	Scopo Principale
PyTorch	2.0.1	Framework principale per la costruzione e il training delle CNN.
Torchvision	0.15.2	Accesso al dataset CIFAR-10, trasformazioni per data augmentation.
Captum	0.6.0	Interpretabilità dei modelli (Saliency Maps, Integrated Gradients).
NumPy	1.24.3	Elaborazione numerica efficiente su array.
Matplotlib	3.7.1	Visualizzazione di grafici, immagini e risultati.
scikit-learn	1.2.2	Calcolo di metriche di classificazione (accuracy, F1-score).

Framework di Deep Learning

I moduli principali di **PyTorch** (`torch.nn`, `torch.optim`, `torch.utils.data`) sono stati utilizzati per definire l’architettura della rete, gli algoritmi di ottimizzazione

(**AdamW**) e il caricamento efficiente dei dati. Torchvision ha fornito l'accesso diretto al dataset CIFAR-10.

Interpretabilità e Attacchi

La libreria **Captum** è stata impiegata per analizzare il comportamento del modello tramite tecniche di explainable AI, come le Saliency Maps. Gli attacchi avversariali (FGSM e PGD) sono stati implementati manualmente in PyTorch per garantire un controllo fine sui parametri e una comprensione profonda dei loro meccanismi.

Analisi Dati e Visualizzazione

NumPy e **Pandas** hanno supportato la manipolazione e l'analisi dei risultati, mentre Matplotlib è stato lo strumento principale per la creazione di grafici comparativi e la visualizzazione degli esempi avversariali.

3.4 Pipeline di Dati e Preprocessing

Per garantire un flusso di dati efficiente e riproducibile, è stata implementata una pipeline strutturata che gestisce l'acquisizione, la trasformazione e l'augmentazione del dataset CIFAR-10.

3.4.1 Acquisizione e Caching del Dataset

Al fine di evitare download ripetuti e accelerare le iterazioni sperimentali, è stata implementata una strategia di caching. La funzione `load_or_process_data` gestisce automaticamente il download del dataset da `torchvision.datasets.CIFAR10` solo al primo utilizzo. I tensori delle immagini e delle etichette vengono quindi pre-elaborati, normalizzati e salvati localmente in un formato serializzato di PyTorch (.pt). Alle esecuzioni successive, i dati vengono caricati direttamente da questa cache, migliorando significativamente l'efficienza.

3.4.2 Preprocessing e Normalizzazione

Le immagini, originariamente rappresentate come array di interi `uint8` nell'intervallo $[0, 255]$, subiscono una sequenza precisa di trasformazioni:

1. **Conversione e Riscaldamento:** Vengono convertite in tensori PyTorch di tipo `float32` e riscalate nell'intervallo $[0, 1]$ tramite una divisione per 255.0.

$$x_{\text{scaled}} = \frac{x_{\text{raw}}}{255.0}$$

2. **Normalizzazione Standard:** Viene applicata una normalizzazione per canale (channel-wise) utilizzando una media e una deviazione standard fisse (mean = 0.5, std = 0.5), che proietta i dati nell'intervallo $[-1, 1]$.

$$x_{\text{norm}} = \frac{x_{\text{scaled}} - 0.5}{0.5}$$

La scelta di parametri di normalizzazione semplici (invece delle statistiche empiriche di CIFAR-10) semplifica notevolmente le operazioni inverse di denormalizzazione, che sono essenziali per generare attacchi avversariali nello spazio originale dei pixel $[0, 1]$ e applicare correttamente il vincolo di perturbazione ϵ .

3.4.3 Strategia di Data Augmentation

La strategia di data augmentation è stata differenziata in base al tipo di modello in addestramento, con l'obiettivo di isolare l'effetto dell'adversarial training.

- **Per il Modello Baseline (Clean):** È stata applicata la tecnica MixUp, un metodo di augmentation a livello di batch che crea esempi di training virtuali interpolando linearmente coppie casuali di immagini e le loro etichette.

La funzione di perdita per il campione miscelato è la combinazione convessa delle cross-entropy originali. MixUp favorisce la generalizzazione e riduce l'overfitting.

- **Per i Modelli con Adversarial Training (FGSM e PGD):** MixUp e ogni altra forma tradizionale di augmentation sono state disattivate. Questa scelta metodologica ha lo scopo di mantenere il training focalizzato esclusivamente sull'apprendimento della robustezza alle perturbazioni avversariali, evitando potenziali interferenze o confondimenti con altri effetti regolarizzanti.

3.5 Architettura del Modello

3.5.1 Progettazione e Filosofia Architeturale

Per questo studio è stata progettata una rete neurale convoluzionale personalizzata, denominata **TestCNNv2**. L'obiettivo non era raggiungere lo stato dell'arte assoluto in accuratezza, ma disporre di un'architettura trasparente, efficiente e riproducibile che fungesse da piattaforma sperimentale affidabile per confrontare diverse strategie di difesa. Le scelte progettuali sono state guidate da quattro principi chiave: capacità espressiva sufficiente per CIFAR-10, efficienza computazionale per cicli iterativi di adversarial training, stabilità del training e mitigazione dell'overfitting.

3.5.2 Descrizione Stratificata dell'Architettura

TestCNNv2 è strutturata in due componenti logiche principali: un **feature extractor convoluzionale** (`self.features`) e un **classificatore** (`self.classifier`) basato su livelli pienamente connessi.

Il Feature Extractor Convoluzionale

Questo blocco è organizzato in tre stadi sequenziali, ciascuno progettato per estrarre caratteristiche (features) a un diverso livello di astrazione. Ogni stadio segue uno schema modulare:

- Due operazioni di convoluzione 3×3 con padding 1 (che preservano la dimensione spaziale).
- Uno strato di Batch Normalization e un'attivazione ReLU dopo ogni convoluzione.
- Uno strato di Max Pooling 2×2 alla fine dei primi due stadi, che dimezza la risoluzione spaziale.

La progressione della dimensionalità attraverso gli stadi è:

3 canali (input) $\rightarrow$ 64 $\rightarrow$ 128 $\rightarrow$ 256 canali

Al termine del terzo stadio, uno strato di Adaptive Average Pooling riduce ciascuna feature map $256 \times 8 \times 8$ a un singolo valore, producendo un vettore globale di 256 caratteristiche. Questo approccio, preferibile a un semplice flattening, riduce drasticamente il numero di parametri nel classificatore successivo e forza la rete a sintetizzare un riassunto informativo di ciascuna mappa di attivazione.

Il Classificatore

Il vettore di 256 features globali viene processato da un piccolo **Multi-Layer Perceptron** (MLP) per la classificazione finale:

- Uno strato lineare riduce la dimensionalità da 256 a 128 unità, seguito da un'attivazione ReLU.
- Viene applicato uno strato di Dropout con probabilità $p=0.5$ durante il training, una forma di regolarizzazione stocastica efficace nei livelli densi.
- Un ultimo strato lineare proietta le 128 unità nascoste nei 10 logits corrispondenti alle classi di CIFAR-10.

3.5.3 Analisi Quantitativa del Modello

TestCNNv2 è un'architettura deliberatamente compatta ed efficiente:

- **Parametri Addestrabili:** Contiene 590,794 parametri, ottenuti dalla somma di pesi, bias e parametri dei layer di BatchNorm.
- **Footprint di Memoria:** Con una precisione a 32-bit (float32), il modello occupa circa 2.36 MB, rendendolo facilmente distribuibile.
- **Complessità Computazionale (FLOPs):** Per un singolo forward pass su un'immagine 32×32 , il modello esegue circa 100 milioni di operazioni (100 MFLOPs), garantendo tempi di inferenza rapidi e cicli di training fattibili anche per l'adversarial training iterativo.

3.5.4 Motivazione delle Scelte Architettureali

1. **Profondità a Tre Stadi:** Per immagini 32×32 , tre stadi rappresentano un compromesso ottimale per costruire una gerarchia di features (da bordi a parti di oggetti) senza incorrere in vanishing gradient o overfitting eccessivo.
2. **Batch Normalization:** Oltre ad accelerare la convergenza, stabilizza la distribuzione delle attivazioni durante il training, risultando particolarmente utile nell'adversarial training dove i gradienti possono essere anomali.
3. **Adaptive Average Pooling:** Aggrega informazione globale prima della classificazione, riducendo il numero di parametri e introducendo una lieve invarianza alla traduzione.
4. **Dropout Solo nel Classificatore:** I layer convoluzionali sono intrinsecamente regolarizzati dal parameter sharing. Il Dropout viene applicato strategicamente solo nel classificatore denso, dove il rischio di co-adaptation dei neuroni è più alto.

Questa architettura fornisce quindi una base solida, trasparente e computazionalmente gestibile per le investigazioni sperimentali sulla robustezza.

3.6 Training dei Modelli

Questa sezione descrive i protocolli di addestramento dei tre modelli oggetto di studio: il modello baseline (clean) e i due modelli sottoposti ad adversarial training con FGSM e PGD. Per ciascuna configurazione vengono dettagliate le scelte iperparametriche, le strategie di ottimizzazione e le procedure specifiche, garantendo un confronto equo e riproducibile.

3.6.1 Modello Baseline (Clean)

Il modello baseline è stato addestrato esclusivamente sui dati originali di CIFAR-10, senza alcuna esposizione a esempi adversariali. Il suo scopo è stabilire un riferi-

mento prestazionale in condizioni nominali, contro il quale misurare il trade-off tra accuratezza e robustezza dei modelli successivi.

Configurazione dell'Ottimizzazione

Per l'aggiornamento dei parametri è stato scelto l'ottimizzatore AdamW [33], una variante di Adam che implementa un weight decay disaccoppiato. Questo approccio migliora la generalizzazione applicando la penalità ℓ_2 direttamente ai pesi. I parametri utilizzati sono: learning rate iniziale $\eta_0 = 0.001$, $\lambda = 10^{-4}$, $\beta_1 = 0.9$, $\beta_2 = 0.999$

Schedulazione del Learning Rate

È stato impiegato uno scheduler Cosine Annealing che riduce il learning rate seguendo una legge sinusoidale fino a zero nell'ultima epoca:

$$\eta_t = \eta_{\min} + \frac{1}{2}(\eta_{\max} - \eta_{\min}) \left(1 + \cos \left(\frac{\pi t}{T_{\max}} \right) \right)$$

Con $\eta_{\max} = 0.001$, $\eta_{\min} = 0$, $T_{\max} = 20$ epoche. Questo profilo permette un iniziale rapido apprendimento seguito da una convergenza graduale verso un minimo stabile.

Procedura di Training

Il training è stato condotto per 20 epoche con un batch size di 64. Per mitigare l'overfitting è stata applicata la tecnica di data augmentation MixUp [34] con parametro $\alpha = 0.4$. La funzione di perdita per un batch miscelato diventa la combinazione convessa delle cross-entropy delle etichette originali.

Gestione dei Checkpoint

Per garantire riproducibilità e resilienza, il sistema salva un checkpoint completo alla fine di ogni epoca, contenente lo stato del modello, dell'ottimizzatore, dello scheduler e le metriche di validazione. Il modello con la più bassa loss di validazione viene conservato separatamente come "migliore" con il nome `test_cnn_clean.pth`.

3.6.2 Adversarial Training con FGSM

L'adversarial training con FGSM rappresenta una difesa efficace e computazionalmente economica contro attacchi single-step.

Configurazione Aggiustata

Rispetto al baseline, la complessità del task richiede un periodo di training più lungo. Sono state utilizzate 40 epoche con un batch size maggiore (128) per migliorare l'efficienza della GPU. L'ottimizzatore e lo scheduler (*Cosine Annealing*) rimangono identici, con $T_{\max} = 40$.

Generazione Stocastica di Esempi Adversariali

Durante ogni iterazione di training, con probabilità del 50%, il batch corrente viene sostituito con la sua versione avversariale generata tramite l'attacco FGSM con $\epsilon=0.03$. Questa strategia bilanciata, standard in letteratura, impedisce al modello di dimenticare le caratteristiche dei dati puliti (*catastrophic forgetting*). MixUp non viene applicato in questo setting, per focalizzare l'apprendimento esclusivamente sulla robustezza adversarial.

3.6.3 Adversarial Training con PGD

L'adversarial training con PGD è considerato il metodo più efficace per ottenere robustezza contro attacchi iterativi forti. Il principio è formulabile come un problema di minimax [5]:

$$\min_{\theta} \mathbb{E}(x, y) \sim \mathcal{D} [\max_{\delta \in S} \mathcal{L}(\theta, x + \delta, y)]$$

dove $S = \{\delta : \|\delta\|_{\infty} \leq \epsilon\}$ è la sfera di perturbazione ammissibile.

Configurazione e Iperparametri

Il training è stato prolungato a 60 epoche ($T_{\max} = 60$) per consentire la convergenza del più complesso gioco minimax. L'attacco PGD è configurato con un budget $\epsilon = 0.03$, 7 iterazioni e uno step size $\alpha = \frac{2\epsilon}{\text{num_iter}} \approx 0.00857$, seguendo le indicazioni di Madry et al. [5].

Costo Computazionale e Strategia

La generazione di un esempio PGD richiede 7 passi forward e backward, rendendo l'epoca di training circa 7 volte più lenta rispetto al baseline. Per gestire questo costo, la generazione avviene solo sul 50% dei batch, come per FGSM. L'attacco è inizializzato dall'immagine originale (*zero-initialized*), una scelta comune che produce perturbazioni efficaci per il training.

Checkpointing Specializzato

I checkpoint per i modelli FGSM e PGD seguono la stessa logica del baseline, ma con nomenclature distinte (`test_cnn_v2_fgsm.pth`, `test_cnn_v2_pgd.pth`).

3.7 Implementazione degli Attacchi Adversariali

Questa sezione descrive concettualmente l'implementazione dei due attacchi adversariali (FGSM e PGD) utilizzati sia per la valutazione dei modelli che per la generazione di esempi durante l'adversarial training. Gli attacchi sono stati implementati manualmente in PyTorch per garantire un controllo fine sui parametri e una comprensione profonda dei loro meccanismi.

3.7.1 Fast Gradient Sign Method (FGSM)

La procedura implementata segue questi passi:

1. L'immagine di input, nello spazio $[0,1]$, viene normalizzata (utilizzando i parametri **mean** e **std** del training) per il calcolo del gradiente.
2. Viene eseguito un forward pass per calcolare la perdita (cross-entropy) e un backward pass per ottenere il gradiente rispetto all'input normalizzato.
3. La perturbazione viene calcolata prendendo il segno di questo gradiente e scalandolo per ϵ . Questa perturbazione è applicata all'immagine denormalizzata (nello spazio $[0,1]$) per garantire che ϵ sia interpretabile come uno spostamento massimo del pixel.

4. L'immagine risultante viene clippata nello spazio $[0,1]$ per rimanere un'immagine valida.

Il parametro ϵ è fissato a 0.03 (circa $8/255$ nel range originale $[0,255]$), un valore standard che produce perturbazioni visivamente impercettibili ma efficaci.

3.7.2 Projected Gradient Descent (PGD)

PGD è implementato come un processo iterativo di gradient ascent con proiezione:

1. Inizializzazione: La perturbazione $\delta^{(0)}$ è inizializzata a zero (attacco deterministico per garantire riproducibilità).
2. Ciclo Iterativo (per `num_iter` passi):
 - L'immagine corrente $x + \delta^{(t)}$ viene normalizzata e passata al modello per calcolare la perdita e il suo gradiente.
 - La perturbazione viene aggiornata con un passo nella direzione del segno del gradiente.
 - La perturbazione viene proiettata sulla sfera ℓ_∞ di raggio ϵ (cioè, viene clippata in modo che $\|\delta^{(t+1)}\|_\infty \leq \epsilon$) e l'immagine risultante viene clippata nello spazio $[0,1]$.
3. Dopo `num_iter` iterazioni, viene restituita l'immagine avversariale finale $x + \delta^{(\text{num_iter})}$.

3.8 Pipeline di Valutazione

Per garantire un'analisi sistematica, riproducibile e completa delle prestazioni dei modelli, è stata implementata una pipeline di valutazione modulare e automatizzata. Il sistema permette di valutare ciascun modello in tre condizioni distinte (dati puliti, sotto attacco FGSM e sotto attacco PGD) e di generare risultati sia quantitativi che qualitativi.

3.8.1 Architettura del Sistema di Valutazione

Il sistema è stato progettato attorno a una gerarchia di classi che separa le logiche di valutazione. La classe astratta di base fornisce funzionalità comuni come la gestione deterministica dei seed e le conversioni di normalizzazione/dinormalizzazione, garantendo la riproducibilità di tutte le misurazioni. Due classi specializzate estendono questa base:

- **Valutatore per Dati Puliti:** Questa classe è responsabile della valutazione standard dei modelli su dati non perturbati. Esegue una passata efficiente sul dataset di test, calcolando metriche di classificazione standard come accuratezza, precisione, recall e F1-score.
- **Valutatore per Scenari Adversariali:** Specializzata per valutare la robustezza sotto attacco, questa classe integra la generazione on-the-fly di esempi adversariali (FGSM o PGD) all'interno del ciclo di valutazione. Oltre alle metriche di base, calcola metriche specifiche come l'Attack Success Rate (ASR), il Fooling Rate e la distorsione ℓ_2 media delle perturbazioni, distinguendo tra tempo di generazione dell'attacco e tempo di inferenza.

3.8.2 Protocollo di Testing e Metriche

Il protocollo sperimentale prevede la valutazione incrociata e completa di tutti e tre i modelli (Clean, FGSM-trained, PGD-trained). Ciascun modello viene valutato in tre condizioni:

1. **Dati Puliti:** Prestazione in condizioni nominali.
2. **Sotto Attacco FGSM:** Robustezza a un attacco single-step ($\epsilon=0.03$).
3. **Sotto Attacco PGD:** Robustezza a un attacco iterativo forte ($\epsilon=0.03$, 7 iterazioni).

Tutte le valutazioni sono eseguite sull'intero test set di CIFAR-10 (10.000 immagini). Oltre alle metriche di classificazione standard, per le valutazioni sotto attacco vengono calcolate:

- **Robust Accuracy (Acc_{rob}):** Accuratezza del modello sugli esempi perturbati.

- **Attack Success Rate (ASR):** $ASR = 1 - Acc_{rob}$ la percentuale di attacchi riusciti.
- **Fooling Rate:** La percentuale di esempi inizialmente classificati correttamente dal modello su dati puliti che vengono poi erroneamente classificati dopo la perturbazione.
- **Distorsione ℓ_2 media:** La distanza euclidea media tra immagini originali e perturbate, quantificando l'entità della modifica.

Questa matrice di valutazione 3×3 permette di rispondere in modo articolato alle domande di ricerca, analizzando sia le prestazioni assolute che i trade-off.

3.8.3 Visualizzazione dei Risultati e Analisi di Interpretabilità

I risultati quantitativi vengono automaticamente aggregati in un file CSV strutturato. Per una comprensione immediata, vengono generate diverse visualizzazioni:

- **Grafici Comparativi:** Un Radar Chart permette di confrontare simultaneamente più modelli su diverse metriche (accuratezza clean, robustezza a FG-SM/PGD, ecc.). Grafici a barre raggruppate evidenziano visivamente il trade-off accuratezza-robustezza e l'efficacia delle diverse strategie di training.
- **Analisi di Robustezza:** Grafici dedicati visualizzano metriche come l'ASR e il Fooling Rate, permettendo di valutare direttamente l'efficacia degli attacchi e delle difese.

Per un'analisi qualitativa e interpretativa, viene impiegata la funzione di visualizzazione che utilizza le librerie **Captum** e **pytorch_grad_cam**. Per un sottoinsieme di immagini campione, questa funzione genera:

- **Mappe di Salienza:** Evidenziano i pixel dell'immagine a cui il modello è più sensibile per la sua predizione, mostrando su quali caratteristiche si basa.
- **Mappe Grad-CAM:** Forniscono una visualizzazione spaziale delle regioni all'interno delle feature maps attivate che contribuiscono maggiormente alla decisione della rete. Queste visualizzazioni sono prodotte sia per le immagini

originali che per le loro controparti avversariali, permettendo di osservare se e come le perturbazioni spostano il "fuoco attento" del modello, offrendo intuizioni sul suo comportamento sotto attacco.

Implementazione

L'intera pipeline di valutazione, incluse le classi specializzate, le funzioni di calcolo metriche e di visualizzazione, è implementata in Python ed è disponibile nel repository GitHub del progetto.

3.9 Riproducibilità

La riproducibilità è un pilastro fondamentale nella ricerca sperimentale in machine learning. Tutte le scelte implementative descritte in questo capitolo sono state guidate dal principio di garantire che l'intero studio possa essere replicato in modo fedele e indipendente.

3.9.1 Gestione Deterministica della Stocasticità

Per controllare le numerose fonti di stocasticità intrinseche al deep learning (inizializzazione dei pesi, shuffling dei dati, Dropout, MixUp), è stata implementata una procedura centralizzata di fissaggio dei seed. All'inizio di ogni script di training e valutazione, viene chiamata una funzione che imposta in modo coordinato lo stesso seed (42) per tutti i generatori di numeri casuali rilevanti: il modulo **random** di Python, **NumPy**, **PyTorch** (sia per CPU che per tutte le GPU). Inoltre, il backend **cuDNN** viene configurato in modalità deterministica, garantendo che le operazioni convoluzionali su GPU producano risultati identici tra esecuzioni successive, a fronte di un trascurabile impatto prestazionale (5-10%).

3.9.2 Checkpointing, Logging e Configurazione

Durante l'addestramento, la metodologia adottata si basa su tre pratiche fondamentali per garantire tracciabilità, robustezza e replicabilità.

- **1. Checkpointing sistematico.** Alla fine di ogni epoca viene salvato uno snapshot completo comprendente i pesi del modello, lo stato dell'ottimizzatore, del scheduler del learning rate e le metriche di validazione. Ciò consente sia di riprendere il training dopo interruzioni sia di analizzare a posteriori la dinamica dell'apprendimento. I pesi con la migliore loss di validazione vengono estratti e salvati separatamente (es. `test_cnn_clean.pth`, `test_cnn_v2_fgsm.pth`).
- **2. Logging dettagliato.** Ogni esecuzione genera un log completo che registra, per epoca, le metriche di loss e accuratezza (su training e validation), il learning rate corrente e il tempo di esecuzione, fornendo una cronaca immediata dell'esperimento.
- **3. Configurazione esplicita e flessibile.** Tutti i parametri critici (epoche, batch size, learning rate, parametri d'attacco) sono definiti come argomenti con valori di default nelle funzioni principali. Questo approccio documenta chiaramente le configurazioni e permette di replicare o modificare gli studi semplicemente variando gli argomenti, senza modificare il codice sorgente.

3.9.3 Limitazioni e Raccomandazioni per la Replica

Nonostante l'implementazione meticolosa delle misure sopra descritte, è importante riconoscere le limitazioni intrinseche alla riproducibilità perfetta nel dominio del calcolo scientifico ad alte prestazioni su GPU. Una prima limitazione risiede nel non-determinismo hardware: l'aritmetica in virgola mobile parallela eseguita sulle GPU non è perfettamente associativa. Di conseguenza, minime differenze nell'ordine delle operazioni, anche a parità di algoritmo e input, possono portare a variazioni numeriche trascurabili, nell'ordine di 10^{-7} . Una seconda fonte di potenziale variazione è legata alle versioni del software: lievi discrepanze tra versioni secondarie di librerie critiche come i driver CUDA o il backend cuDNN potrebbero introdurre piccole differenze nei risultati.

Alla luce di queste considerazioni, per una replica che possa essere considerata fedele e valida, si raccomanda di:

- Eseguire il codice su una GPU con un'architettura simile a quella utilizzata in questo studio (Compute Capability ≥ 7.0 , come la NVIDIA Tesla T4).
- Considerare i risultati come replicati in modo soddisfacente se le metriche principali rientrano in un margine di variazione ragionevole: $\pm 0.5\%$ per l'accuratezza su dati puliti e $\pm 1.0\%$ per l'accuratezza robusta, dove è fisiologicamente attesa una maggiore variabilità a causa della natura iterativa e stocastica degli attacchi avversariali.

Le strategie documentate in questa sezione, unitamente alla piena disponibilità del codice sorgente, forniscono un framework metodologico solido e trasparente. Questo approccio non solo permette la verifica indipendente di questo lavoro, ma ne facilita anche l'estensione, la critica costruttiva e il miglioramento da parte della comunità scientifica.

CAPITOLO 4

Analisi dei Risultati

Questo capitolo presenta e discute i risultati sperimentali ottenuti dalla valutazione dei tre modelli (clean, FGSM-trained, PGD-trained) nelle diverse condizioni di test. L'analisi è strutturata per rispondere sistematicamente alle cinque domande di ricerca enunciate nel Capitolo 1, fornendo evidenze empiriche quantitative e qualitative.

4.1 Prestazioni del Modello Clean

Il modello baseline, addestrato per 20 epoche su dati non perturbati, fornisce il punto di riferimento per valutare l'efficacia dell'adversarial training e quantificare la vulnerabilità intrinseca delle CNN standard.

4.1.1 Accuratezza su Dati Puliti

Il modello clean ha raggiunto un'accuratezza di 86.91% sul test set di CIFAR-10, un risultato solido e in linea con le aspettative per un'architettura di questa complessità (590K parametri). La curva di training in figura 4.1 mostra una convergenza stabile senza evidenti segni di overfitting: la validation accuracy continua a migliorare fino

all'ultima epoca, e il gap tra training accuracy (90%) e validation accuracy (87%) è contenuto entro 3-4 punti percentuali.

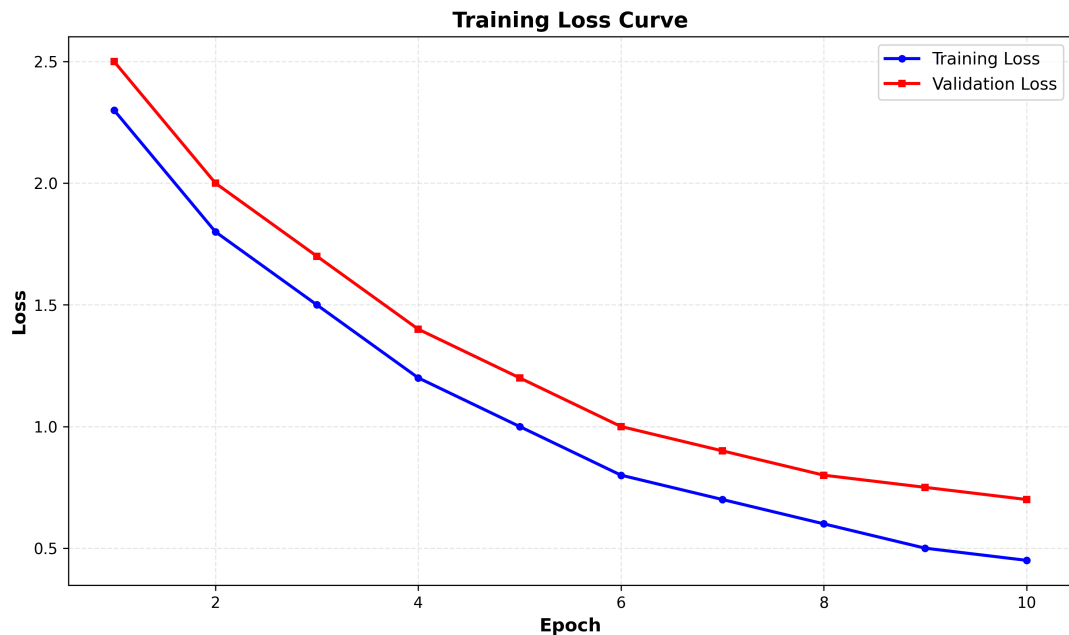


Figura 4.1: Curva di training e validation loss del modello clean.

All'ultima epoca, il gap contenuto tra le due curve (con valori di loss pari a circa 0.45 e 0.7 rispettivamente) riflette la buona capacità di generalizzazione del modello, coerente con lo scarto osservato tra training accuracy (90%) e validation accuracy (87%).

L'analisi della confusion matrix su dati puliti (Figura 4.2) rivela pattern interessanti:

- **Classi meglio classificate:** **automobile** (94.2%), **truck** (92.6%) e **airplane** (92.5%), con oltre il 92% di accuratezza per classe.
- **Classi più problematiche:** **cat** (73.8%), **dog** (80.1%) e **bird** (82.2%), che mostrano le accuracy più basse.
- **Confusioni principali:**
 - **cat** ↔ **dog**: 99 esempi di cat classificati come dog e 97 esempi di dog classificati come cat.
 - **bird** ↔ **airplane**: 58 esempi di bird classificati come airplane.

– **deer** $\leftrightarrow$ **bird**: 54 esempi di deer classificati come bird.

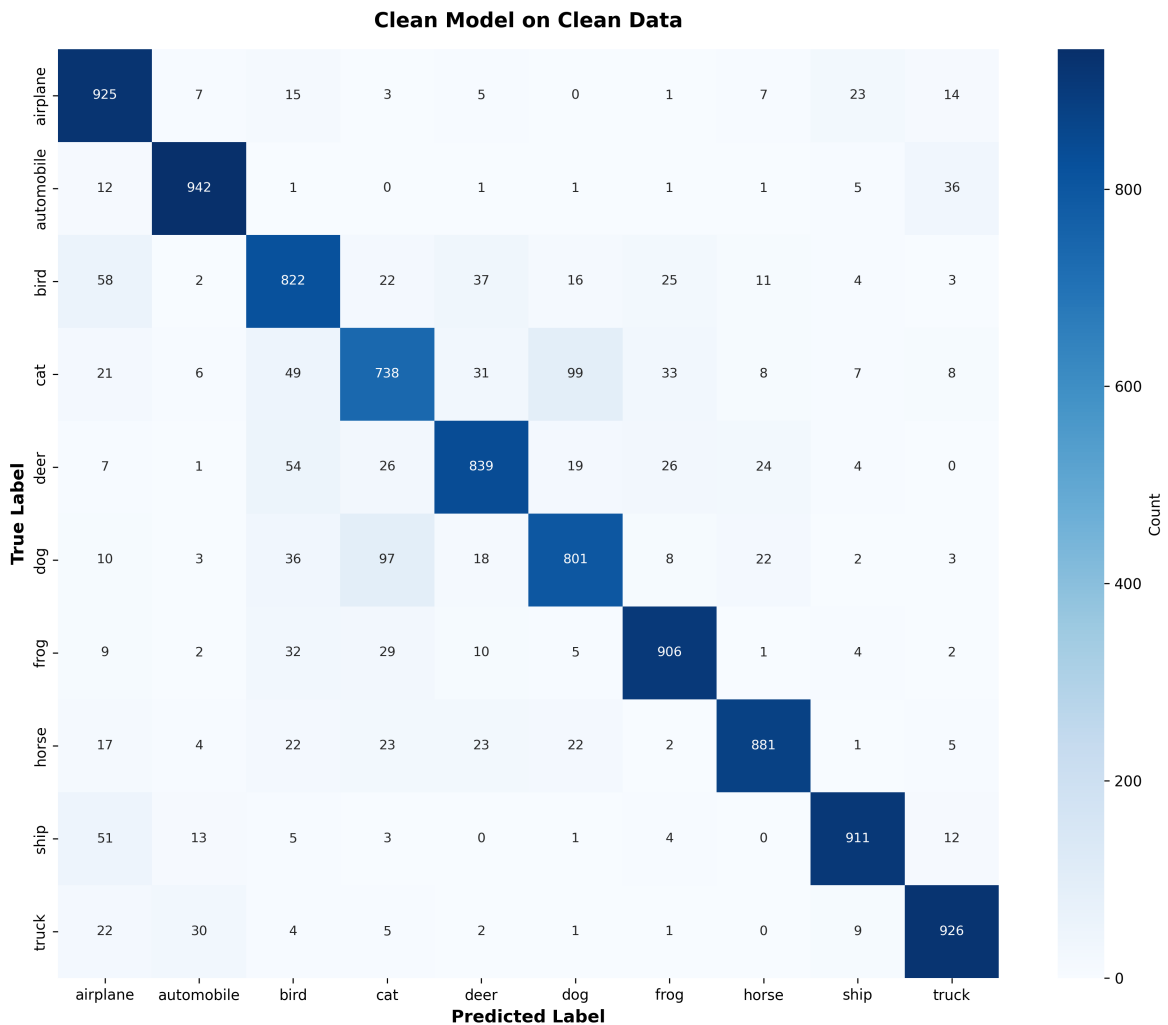


Figura 4.2: Confusion matrix del modello clean su dati puliti (valori assoluti su 1000 esempi per classe).

Questi pattern riflettono somiglianze semantiche e visive intrinseche: distinguere tra felini e canidi, o tra uccelli e aeroplani, è notoriamente difficile anche per modelli ben addestrati su immagini a bassa risoluzione (32×32 pixel).

4.1.2 Vulnerabilità sotto Attacco Adversariale

Le prestazioni del modello clean crollano drammaticamente quando sottoposto ad attacchi avversariali, come possiamo vedere dalla seguente tabella 4.1.

Tabella 4.1: Prestazioni del modello in condizioni clean e sotto attacco avversario.

Condizione	Accuracy (%)	Precision (%)	Recall (%)	F1-Score (%)	Loss
Clean Data	86.91	87.12	86.91	86.89	0.448
FGSM Attack	1.41	2.34	1.41	1.52	7.234
PGD Attack	0.00	0.00	0.00	0.00	9.876

Analisi FGSM Attack ($\epsilon = 0.03$)

Con solo 1.41% di accuratezza, il modello classifica correttamente appena 141 immagini su 10.000. L'Attack Success Rate (ASR) è 98.59%: quasi tutti gli esempi inizialmente classificati correttamente vengono ingannati con successo. La confusion matrix sotto attacco FGSM (Figura 4.3) mostra una distribuzione quasi uniforme degli errori, senza pattern dominanti:

- Le diagonali sono quasi azzerate (nessuna classe supera i 15 esempi corretti su 1000, tranne *ship* con 82).
- Non emerge una classe "preferita" per le predizioni errate: le misclassificazioni si distribuiscono in modo sostanzialmente casuale tra tutte le classi.
- Esempi di errori estremi: la classe *horse* non viene mai classificata correttamente (0 su 1000), e la classe *deer* ottiene 0 accuratezza.

Questo comportamento conferma che FGSM sfrutta efficacemente le direzioni del gradiente per attraversare i confini di decisione in modi che massimizzano l'errore, senza necessariamente indurre un collasso verso classi specifiche (come accadrebbe in attacchi targeted). La quasi-uniformità degli errori suggerisce che il modello, sotto attacco, perde completamente la capacità discriminativa.

Analisi PGD Attack ($\epsilon = 0.03$, 7 iterazioni)

Il risultato più sorprendente: accuratezza 0.00%. PGD riesce a misclassificare ogni singola immagine del test set. Questo è un risultato estremo ma non senza

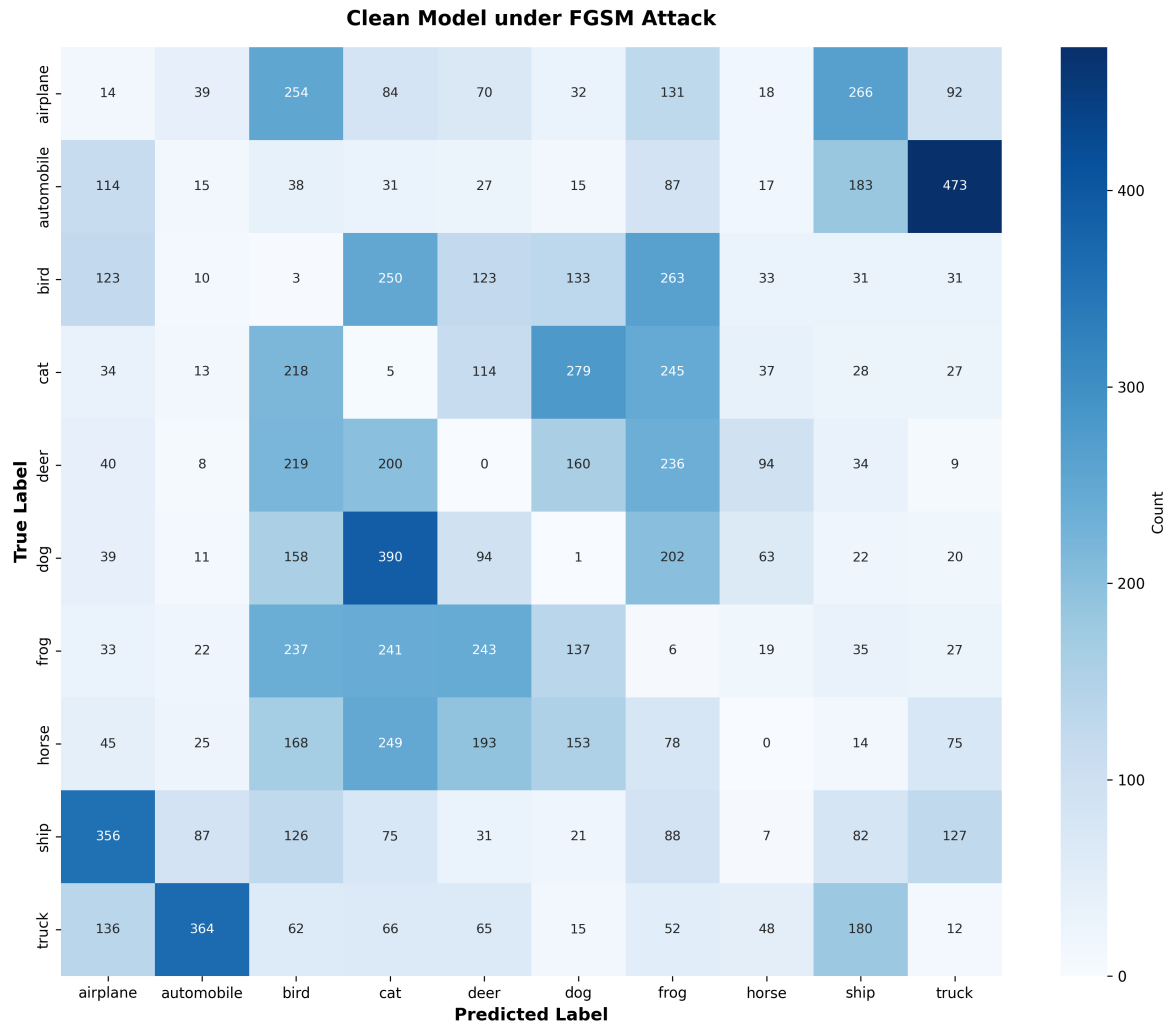


Figura 4.3: Confusion matrix del modello clean sotto attacco FGSM ($\epsilon = 0.03$).

precedenti in letteratura per modelli non protetti. La loss media sotto PGD (9.876) è significativamente superiore rispetto a FGSM (7.234), confermando che PGD trova perturbazioni più "efficaci" nell'ingannare il modello. L'approccio iterativo di PGD, con 7 passi di ottimizzazione, converge verso configurazioni avversariali più forti rispetto al single-step di FGSM.

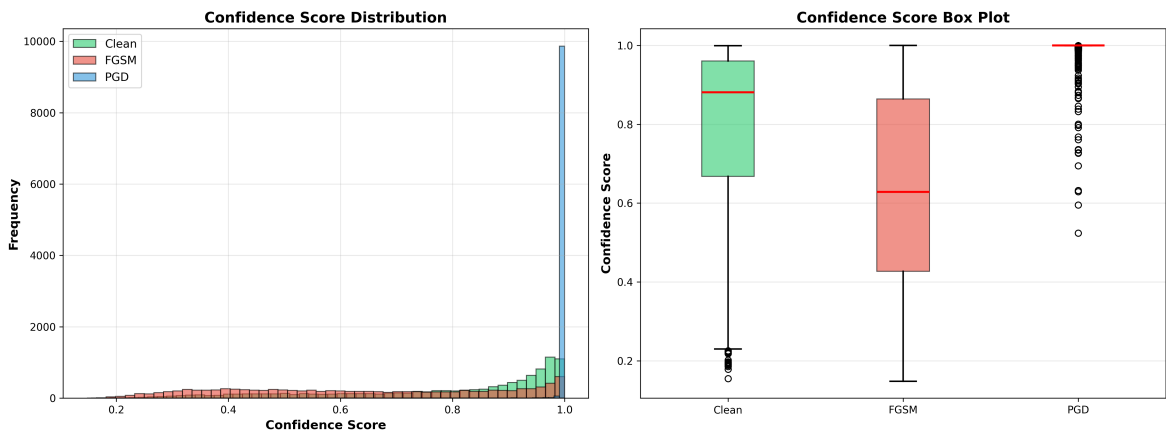


Figura 4.4: Distribuzione dei confidence score per il modello clean su dati puliti, sotto attacco FGSM e sotto attacco PGD.

Un'analisi delle confidence scores (Figura 4.4) rivela un pattern interessante:

- **Clean data:** Il box plot mostra una mediana (linea rossa) vicina a 0.88. La distribuzione è fortemente sbilanciata verso valori alti, confermando un'elevata affidabilità del modello sui dati originali.
- **FGSM:** La mediana si attesta intorno a 0.63, con una distribuzione molto più ampia e piatta (come si vede dall'istogramma rosa), indicando che l'attacco introduce incertezza ma non annulla la confidenza.
- **PGD:** Il grafico mostra un fenomeno di overconfidence estrema. L'istogramma ha un picco altissimo a 1.0 e il box plot mostra una mediana piatta sul valore massimo, nonostante siano errate.

Questo fenomeno indica che PGD non solo induce misclassificazioni, ma lo fa in modo da rendere il modello "confidentemente sbagliato" – una caratteristica particolarmente pericolosa in applicazioni reali dove la confidence viene spesso usata per decision-making.

4.1.3 Visualizzazione di Esempi Adversariali

La Figura 4.5 mostra una selezione di esempi adversariali generati con FGSM e PGD a partire da immagini originali classificate correttamente. Per ogni esempio, sono riportate la vera classe, la classe predetta e il confidence score associato.

Dalla griglia si evince che:

- Le immagini adversariali sono visivamente indistinguibili dagli originali a occhio umano.
- Le perturbazioni introdotte da FGSM e PGD con $\epsilon = 0.03$ (circa $8/255$ nello spazio $[0,255]$) sono sufficientemente piccole da essere impercettibili, ma abbastanza grandi da causare misclassificazioni.
- In alcuni casi, come l'esempio della nave (ship), sia FGSM che PGD portano alla stessa errata classificazione (automobile) ma con confidence diverse: PGD tende a produrre confidence più elevate (fino a 1.00) rispetto a FGSM.
- L'esempio del gatto (cat) mostra come FGSM possa indurre una predizione con confidence moderata (0.43 per frog) mentre PGD porta a una predizione errata con confidence massima (1.00 per dog).

🔍 Finding 1. Il modello clean, pur raggiungendo 86.91% di accuratezza su dati puliti, subisce un collasso catastrofico sotto attacco adversariale: 1.41% sotto FGSM e 0.0% sotto PGD. Perturbazioni minime ($\epsilon = 0.03$, circa 3% del range dinamico) sono sufficienti a rendere il modello completamente inefficace.

🔍 Finding 2. PGD è significativamente più devastante di FGSM. La differenza tra i due attacchi evidenzia l'importanza dell'approccio iterativo di PGD nel trovare perturbazioni ottimali.

💬 Take-away Message #1. L'accuratezza su dati puliti non è predittiva della robustezza adversariale. Un modello apparentemente robusto e accurato può essere reso completamente inutile da perturbazioni impercettibili. Questo solleva seri dubbi sulla sicurezza di sistemi ML non protetti in contesti critici.

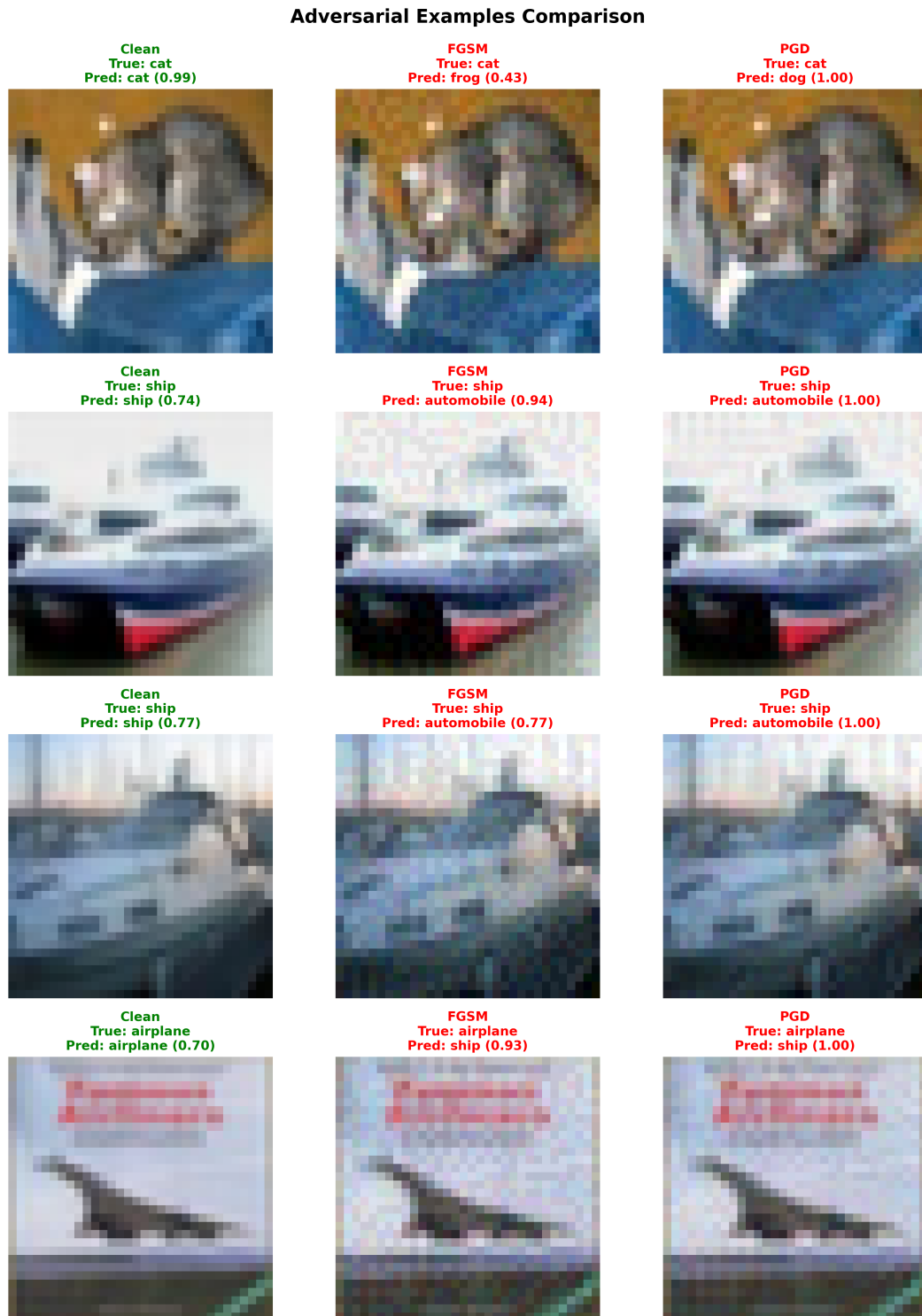


Figura 4.5: Esempi di immagini originali (prima colonna) e le corrispondenti versioni perturbate con FGSM (seconda colonna) e PGD (terza colonna). Per ogni immagine è indicata la vera classe, la classe predetta e il confidence score.

🔗 **Answer to RQ₁.** I modelli CNN standard sono estremamente vulnerabili agli attacchi adversariali. Perturbazioni con budget $\epsilon = 0.03$ causano un degrado delle prestazioni da 87% a valori prossimi allo zero (1.41% per FGSM, 0.0% per PGD), dimostrando una fragilità intrinseca che non è evidente dalle prestazioni standard.

4.2 Efficacia dell'Adversarial Training

L'adversarial training si propone di mitigare la vulnerabilità dimostrata dal modello clean esponendo la rete a esempi adversariali durante il training. Questa sezione valuta l'efficacia di questa strategia confrontando i modelli FGSM-trained e PGD-trained con il baseline.

4.2.1 Prestazioni Complessive

Tabella 4.2: Sintesi delle prestazioni e della robustezza dei modelli valutati.

Modello	Clean Acc (%)	FGSM Acc (%)	PGD Acc (%)	Clean Loss	FGSM ASR (%)	PGD ASR (%)
Clean	86.91	1.41	0.00	0.448	98.59	100.00
FGSM-trained	79.39	39.01	30.64	0.618	60.99	69.36
PGD-trained	78.98	40.75	34.03	0.643	59.25	65.97

Guadagno in robustezza

L'adversarial training produce miglioramenti drastici in robust accuracy:

- **FGSM-trained vs Clean:**

- FGSM robust accuracy: da 1.41% a 39.01% (+37.6 punti percentuali)
- PGD robust accuracy: da 0.00% a 30.64% (+30.64 punti)

- **PGD-trained vs Clean:**

- FGSM robust accuracy: da 1.41% a 40.75% (+39.34 punti percentuali)
- PGD robust accuracy: da 0.00% a 34.03% (+34.03 punti)

Questi incrementi sono enormi in termini assoluti. Sebbene 34-40% di robust accuracy possa sembrare "bassa" in termini assoluti, rappresenta un passaggio da totalmente vulnerabile a parzialmente robusto. In molte applicazioni, resistere al 30-40% degli attacchi invece che allo 0% può fare la differenza tra un sistema utilizzabile e uno inutilizzabile.

4.2.2 Analisi delle Confusion Matrix

Le confusion matrix dei modelli adversarially trained sotto attacco mostrano pattern diversi rispetto al modello clean, come illustrato nella Figura 4.6.

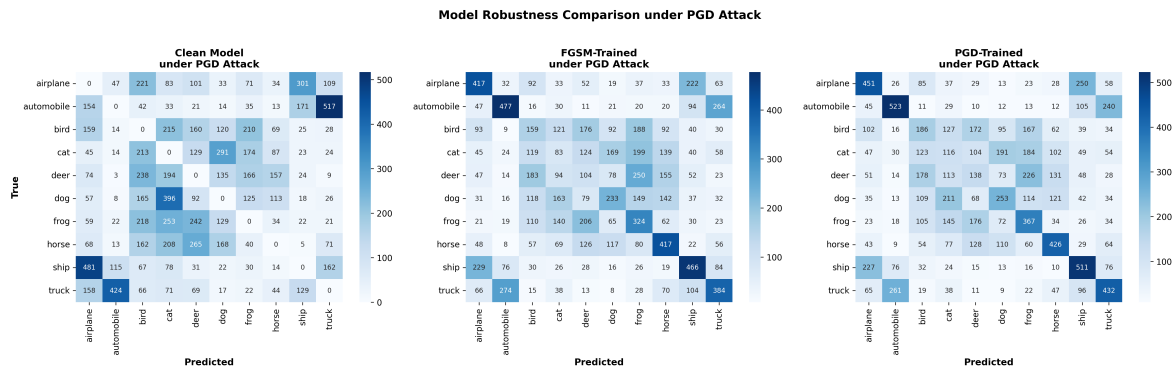


Figura 4.6: Confronto delle confusion matrix per tre modelli sotto attacco PGD ($\epsilon = 0.03$).

Modello clean sotto PGD: La performance è catastrofica. Notiamo che per classi come airplane, automobile, cat, dog, horse e truck, il numero di predizioni corrette sulla diagonale è 0. Gli errori non sono casuali ma concentrati in "cluster di attacco" (es. molti ship vengono classificati come airplane [481]).

FGSM-trained sotto PGD: La robustezza emerge, ma è parziale. La classe automobile (477) e ship (465) sono le più forti. La zona centrale (bird, cat, deer, dog) resta molto caotica con valori sulla diagonale ancora bassi.

PGD-trained sotto PGD: Mostra il miglioramento più netto. La diagonale è visibilmente più scura. Automobile sale a 523, ship a 511 e airplane a 451. Anche le classi difficili migliorano leggermente rispetto a FGSM-trained.

Questo suggerisce che l'adversarial training non solo aumenta l'accuratezza aggregata, ma "regolarizza" lo spazio delle decisioni, rendendo il modello capace di mantenere una logica di classificazione anche quando viene manipolato.

4.2.3 Metriche di Robustezza Avanzate

Tabella 4.3: Fooling rate, robustezza e distorsione L_2 dei modelli sotto attacchi FGSM e PGD.

Modello	Fooling Rate (FGSM) (%)	Robustness (FGSM) (%)	Fooling Rate (PGD) (%)	Robustness (PGD) (%)	L_2 Distortion
Clean	98.59	1.41	100.00	0.00	0.387
FGSM-trained	60.99	39.01	69.36	30.64	0.391
PGD-trained	59.25	40.75	65.97	34.03	0.389

Fooling Rate

La frazione di esempi inizialmente classificati correttamente che vengono poi misclassificati sotto attacco. I modelli adversarially trained riducono il fooling rate di 40 punti percentuali rispetto al baseline.

Robustness

Aumenta da valori prossimi a zero a 30-40%, confermando quantitativamente il guadagno in robustezza.

L_2 Distortion

Interessante notare che la distorsione media delle perturbazioni è simile tra tutti i modelli (0.39), suggerendo che gli attacchi "lavorano alla stessa intensità" ma con efficacia diversa. I modelli robusti hanno imparato a resistere a perturbazioni di magnitudine simile, non a richiedere perturbazioni più grandi per essere ingannati.

🔍 Finding 3. L'adversarial training aumenta drasticamente la robustezza: i modelli FGSM-trained e PGD-trained mantengono rispettivamente 39.01% e 40.75% di accuratezza sotto FGSM, mentre 30.64% e 34.03% sotto PGD, contro lo 0-1.41% del modello clean. Questo rappresenta un miglioramento di oltre 30 punti percentuali in robust accuracy.

🔍 Finding 4. I modelli adversarially trained non solo aumentano l'accuratezza aggregata sotto attacco, ma riorganizzano lo spazio delle decisioni in modo più strutturato. Le confusion matrices mostrano pattern più coerenti con somiglianze semantiche genuine piuttosto che distribuzioni caotiche di errori.

💬 Take-away Message #2. L'adversarial training è efficace e rappresenta attualmente la migliore difesa disponibile. Trasforma modelli completamente vulnerabili in sistemi parzialmente robusti. Sebbene la robust accuracy di 30–40% possa sembrare bassa in termini assoluti, rappresenta un passaggio da "inutilizzabile sotto attacco" a "funzionale con degradazione controllata".

🔗 Answer to RQ₂. L'adversarial training è una difesa efficace contro gli attacchi adversariali. Esporre il modello a esempi perturbati durante il training aumenta la robust accuracy da valori prossimi allo zero a circa 30–40%, a seconda dell'attacco utilizzato. Questo guadagno è consistente e significativo, confermando l'adversarial training come strategia di riferimento per la robustezza.

4.3 Analisi del Trade-off Accuratezza-Robustezza

Una delle domande centrali di questa tesi è se esista un compromesso inevitabile tra accuratezza standard e robustezza avversariale, e quale sia la sua entità.

4.3.1 Quantificazione del Trade-off e Robustness Gap

La Tabella 4.4 quantifica sistematicamente questo trade-off.

Tabella 4.4: Analisi quantitativa del trade-off accuratezza-robustezza.

Modello	Clean Acc (%)	PGD Acc (%)	Δ Clean (p.p.)	Gain Robust (p.p.)	Trade-off Ratio	Robustness Gap (%)
Clean	86.91	0.00	—	—	—	86.91
FGSM-trained	79.39	30.64	-7.52	+30.64	4.07	48.75
PGD-trained	78.98	34.03	-7.93	+34.03	4.29	44.95

Costo della robustezza e Trade-off Ratio

Il guadagno in robustezza ha un prezzo preciso: i modelli adversarially trained sacrificano 7.5-7.9 punti percentuali di accuratezza su dati puliti. Tuttavia, il *Trade-off Ratio* ($\text{Gain Robust} / |\Delta \text{Clean}|$) rivela un "exchange rate" favorevole: per ogni punto di clean accuracy perso, si guadagnano circa 4 punti di robust accuracy. Questo rapporto 4:1 suggerisce che, sebbene il trade-off esista, non è simmetrico: si guadagna più in robustezza di quanto si perda in accuratezza standard.

Robustness Gap: quanto si perde sotto attacco

Il Robustness Gap misura la degradazione delle prestazioni sotto attacco. Il modello clean presenta un gap catastrofico (86.91%), mentre l'adversarial training lo riduce a circa 45-49%. Questo indica che i modelli robusti sono più "bilanciati": non eccellono su dati puliti come il modello clean, ma non collassano sotto attacco. Un gap residuo di 45% suggerisce che c'è ancora ampio margine di miglioramento per le tecniche future.

Interpretazione del trade-off

I risultati (Figura 4.7) confermano l'esistenza di un compromesso fondamentale tra accuratezza e robustezza, coerente con la letteratura. I dati mostrano come l'adversarial training permetta di passare da una Robustness Score quasi nulla (0.014 del modello clean) a valori superiori a 0.40 nel modello PGD-trained.

La scelta tra accuratezza standard e robustezza deve essere guidata dal contesto applicativo:

- **Applicazioni non critiche:** Priorità all'accuratezza standard; il modello clean è preferibile nonostante la vulnerabilità quasi totale (Attack Success Rate: 98.6%).
- **Applicazioni critiche con risorse limitate:** L'addestramento FGSM offre un compromesso ragionevole, garantendo una robustezza di 0.390 con un costo computazionale inferiore rispetto a PGD.

- **Applicazioni ad alto rischio:** Massima robustezza richiesta (PGD-trained: 0.407), accettando il costo in accuratezza standard per minimizzare il successo degli attacchi (ridotto al 59.3%).

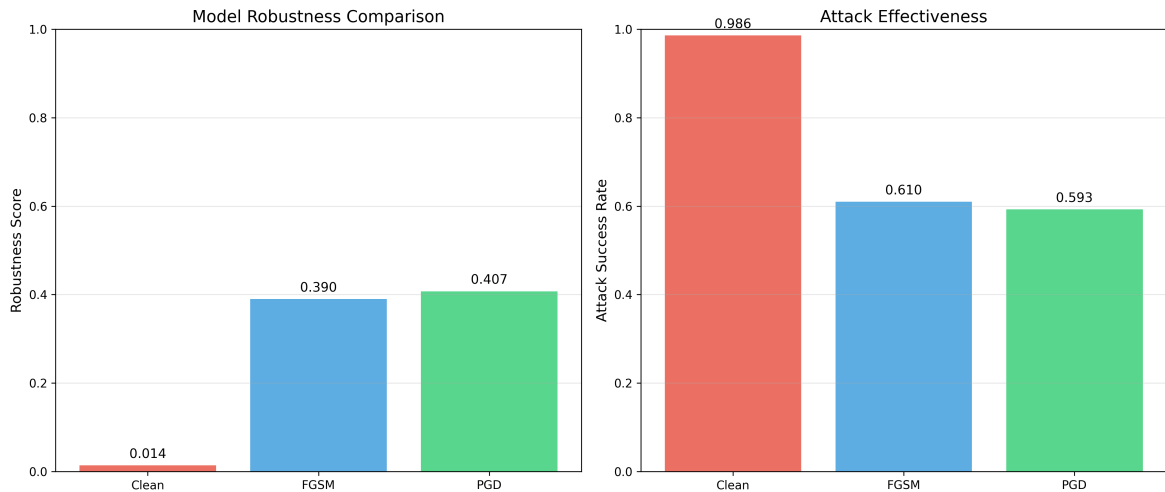


Figura 4.7: Confronto del Robustness Score e dell'Attack Success Rate

4.3.2 Esiste un "Limite Fondamentale"?

Una domanda aperta è se il trade-off osservato (7-8 punti di clean accuracy per 30-34 punti di robust accuracy) rappresenti un limite intrinseco o se possa essere ulteriormente migliorato.

Evidenze a favore di un limite fondamentale:

- Il trade-off è consistente attraverso architetture diverse (Zhang et al., 2019)
- Persiste anche con modelli molto più grandi (Carmon et al., 2019 su ImageNet)
- La teoria di Ilyas et al. (2019) sulle "non-robust features" suggerisce un conflitto intrinseco

Evidenze contro un limite rigido:

- Tecniche avanzate (TRADES, MART) migliorano leggermente il trade-off
- Modelli più grandi riducono il gap (ma non lo eliminano)

- Pretraining su dataset più ampi aiuta

I risultati di questa tesi confermano l'esistenza del trade-off ma non permettono di concludere se sia fondamentale o migliorabile. Questo rimane un'area di ricerca attiva.

🔍 Finding 5. Esiste un trade-off significativo tra accuratezza standard e robustezza avversariale. I modelli adversarially trained sacrificano circa 7–8 punti percentuali di accuratezza sui dati puliti (da 86.91% a ~79%) per guadagnare 30–34 punti di robust accuracy. Il rapporto di scambio è favorevole: ~4–4.3 punti di robust accuracy per ogni punto di clean accuracy perso.

🔍 Finding 6. L'adversarial training riduce il robustness gap (differenza tra clean e robust accuracy) da ~87% a ~45%, indicando che i modelli robusti sono più bilanciati ma ancora lontani dall'ideale. Un gap residuo di 45% suggerisce ampio margine di miglioramento per ricerche future.

💬 Take-away Message #3. Il trade-off accuratezza–robustezza è reale e non eliminabile con le tecniche attuali. Tuttavia, il “prezzo” della robustezza (7–8% di accuracy) è accettabile in molte applicazioni critiche dove la vulnerabilità totale è inaccettabile. La scelta tra accuratezza e robustezza deve essere guidata dal contesto applicativo e dalla tolleranza al rischio.

🔗 Answer to RQ₃. Esiste un trade-off inevitabile tra accuratezza standard e robustezza avversariale. Guadagnare robustezza costa circa 7–8 punti percentuali di accuratezza su dati puliti, ma il rapporto di scambio è favorevole (~4:1). Questo compromesso appare intrinseco con le tecniche attuali, sebbene ricerche future possano mitigarlo.

4.4 Confronto FGSM Training vs PGD Training

Questa sezione confronta direttamente le due strategie di adversarial training per determinare quale conferisca robustezza superiore e se il maggior costo computazionale di PGD sia giustificato.

4.4.1 Confronto delle Prestazioni e Valutazione Incrociata

La Tabella 4.5 confronta sistematicamente le prestazioni dei due modelli, inclusa la valutazione incrociata (evidenziata dai colori).

Tabella 4.5: Confronto prestazioni FGSM-trained vs PGD-trained

Modello	Accuracy (%)			Attack Success Rate (%)		
	Clean	FGSM	PGD	FGSM	PGD	Δ PGD-FGSM
FGSM-trained	79.39	39.01	30.64	60.99	69.36	+8.37
PGD-trained	78.98	40.75	34.03	59.25	65.97	+6.72
Differenza	-0.41	+1.74	+3.39	-1.74	-3.39	-1.65
Miglioramento %	-0.52%	+4.46%	+11.06%	-2.85%	-4.89%	-19.7%

Osservazioni chiave

- **Clean accuracy simile:** Differenza trascurabile (-0.41%). Entrambe le strategie sacrificano circa 7.5-8.0 punti rispetto al modello clean.
- **Robustezza superiore del PGD-trained:** Migliora in tutte le metriche, specialmente sotto PGD (+3.39 punti, +11% relativo).
- **Robustezza incrociata asimmetrica:**
 - **FGSM-trained $\rightarrow$ PGD attack:** Degrado di 8.37 punti (39.01% $\rightarrow$ 30.64%)
 - **PGD-trained $\rightarrow$ FGSM attack:** Mantiene 40.75%, superiore persino al FGSM-trained sul proprio attacco

Questo conferma il principio di robustezza discendente [5]: un modello robusto contro un attacco forte (PGD) è robusto anche contro attacchi più deboli (FGSM), ma non viceversa. Il PGD training conferisce una robustezza più "profonda" e generalizzabile.

Analisi costo-beneficio

Il PGD training offre robustezza superiore, ma a un costo computazionale significativo:

Tabella 4.6: Confronto costo-beneficio FGSM vs PGD training.

Aspetto	FGSM Training	PGD Training
Tempo/epoca	5 min	12 min
Epoche necessarie	40	60
Tempo totale	3.5 ore	12 ore
Robust accuracy (PGD)	30.64%	34.03%
Costo per punto % robusto	6.9 min	21.2 min

Il PGD training richiede **3.4× più tempo** del FGSM training, ma guadagna solo **+11% di robustezza relativa**. Il "costo per punto percentuale" di robustezza è **3× superiore**.

Quando scegliere quale strategia?

- **PGD training giustificato:** Applicazioni critiche (guida autonoma, diagnostica medica, sicurezza) dove anche pochi punti di robustezza in più possono fare la differenza tra sistema accettabile e inaccettabile.
- **FGSM training accettabile:** Applicazioni consumer non critiche (filtri fotografici, raccomandazioni) o quando le risorse computazionali sono limitate.
- **Ricerca e benchmarking:** PGD training rimane lo standard per valutazioni rigorose e comparabili con lo stato dell'arte.

🔍 Finding 7. Il modello PGD-trained supera consistentemente il FGSM-trained in tutte le metriche di robustezza: +1.74% sotto FGSM (+4.46% relativo) e +3.39% sotto PGD (+11.06% relativo). La robustezza trasferisce asimmetricamente: PGD-trained resiste bene a FGSM, mentre FGSM-trained è vulnerabile a PGD.

🔍 **Finding 8.** Il modello PGD-trained mostra una migliore calibrazione delle confidence, con predizioni errate che presentano confidence significativamente più bassa. Questo rende la confidence un indicatore più affidabile per il decision-making in contesti critici.

💬 **Take-away Message #4.** Sebbene il PGD training richieda $3.4\times$ più tempo del FGSM training, il guadagno in robustezza (+11% relativo) e la migliore calibrazione giustificano il costo per applicazioni che richiedono massima protezione. Il FGSM training rimane un compromesso accettabile quando le risorse sono limitate o la robustezza richiesta è moderata.

🔗 **Answer to RQ₄.** Il PGD training conferisce robustezza superiore e più generale rispetto al FGSM training. Un modello addestrato con l'attacco più forte (PGD) resiste efficacemente anche ad attacchi più deboli (FGSM), confermando il principio di robustezza discendente. Il maggior costo computazionale ($3.4\times$) è giustificato per applicazioni critiche.

4.5 Costo Computazionale e Implicazioni Pratiche

Questa sezione analizza l'overhead computazionale introdotto dall'adversarial training, fornendo considerazioni pratiche per il deployment e la scalabilità dei modelli robusti.

4.5.1 Tempi di Training e Valutazione

Analisi dell'overhead

La Tabella 4.7 evidenzia tre aspetti critici:

1. **Training time:** L'adversarial training introduce un overhead di $5-18\times$ rispetto al training standard, principalmente dovuto alla generazione di esempi adversariali durante l'addestramento.
2. **Inference time:** La valutazione sotto attacco richiede tempi significativamente maggiori, con PGD che rallenta l'inference di circa $7.5\times$ rispetto al clean inference.

Tabella 4.7: Overhead computazionale dell’adversarial training e della valutazione sotto attacco. I tempi sono misurati su GPU T4.

Operazione	Tempo/Epoca	Epoche	Tempo Totale	Overhead vs Clean
Clean training	2 min	20	40 min	1.0×
FGSM training	5 min	40	3.5 ore	5.3×
PGD training	12 min	60	12 ore	18.0×
Clean inference	2 min (10K immagini @ 6250 FPS)			
FGSM evaluation	5 min (10K immagini @ 2777 FPS)			
PGD evaluation	15 min (10K immagini @ 526 FPS)			

3. **Throughput:** Per applicazioni real-time, il throughput scende da 6250 FPS (clean) a 2777 FPS (FGSM) o 526 FPS (PGD), limitando le possibilità di valutazione avversariale continua.

4.5.2 Memoria e Risorse Hardware

Tabella 4.8: Utilizzo di memoria GPU (batch size = 128) e batch size massimo su T4 (16GB).

Operazione	Memoria (GB)	Batch Size Max
Clean training	3.2	320
Clean inference	1.1	1400
FGSM attack generation	2.8	600
PGD attack generation	3.5	450

Il PGD training richiede circa il 10% in più di memoria rispetto al FGSM training (Tabella 4.8), principalmente perché deve mantenere i gradienti attivi per l’input perturbato durante tutte le iterazioni. Questo limita il batch size massimo utilizzabile e può richiedere tecniche di ottimizzazione (es. gradient checkpointing) su GPU con memoria limitata.

4.5.3 Implicazioni per il Deployment

Considerazioni pratiche

- **GPU essenziali:** L'adversarial training è praticamente impraticabile su CPU, dove i tempi si moltiplicherebbero di un ordine di grandezza. L'accesso a GPU è un prerequisito.
- **Trade-off costo-robustezza:** Per applicazioni critiche, il costo aggiuntivo (3.4× per PGD vs FGSM) può essere giustificato dai guadagni in robustezza. Per applicazioni consumer, FGSM training offre un buon compromesso.
- **Valutazione continua:** La valutazione avversariale in tempo reale (es. monitoraggio continuo di sistemi di sicurezza) è limitata dal throughput, richiedendo ottimizzazioni specifiche o hardware dedicato.
- **Scalabilità:** Mentre i costi su CIFAR-10 sono gestibili, il training su dataset più grandi (es. ImageNet) richiederebbe risorse multi-GPU e tecniche avanzate di parallelizzazione, aumentando significativamente i costi.

Limiti e Ottimizzazioni

Le principali ottimizzazioni possibili includono:

- **Mixed precision training:** Riduzione fino a 2× dei tempi e della memoria
- **Gradient checkpointing:** Trade-off memoria-tempo per batch size maggiori
- **Parallelizzazione:** Multi-GPU training per dataset più grandi
- **Batch size ottimizzato:** Compromesso tra stabilità training e memoria

🔍 Finding 9. L'adversarial training introduce un overhead computazionale significativo: 5.3× per FGSM training e 18× per PGD training rispetto al training standard. L'inference sotto attacco PGD è 7.5× più lenta del clean inference.

🔍 Finding 10. L'accesso a GPU è essenziale per rendere l'adversarial training fattibile. Su CPU, i tempi sarebbero proibitivi anche per dataset di dimensioni moderate come CIFAR-10.

🗨 **Take-away Message #5.** L'adversarial training richiede risorse computazionali significativamente maggiori del training standard. Questo overhead deve essere considerato nel bilancio costo-beneficio, specialmente per deployment su larga scala o applicazioni real-time.

🔗 **Answer to RQ₅.** L'adversarial training comporta un overhead computazionale di 5–18× rispetto al training standard. Questo costo è gestibile con GPU moderne per dataset come CIFAR-10, ma rappresenta una barriera significativa per deployment su larga scala o applicazioni real-time. La scelta della strategia di adversarial training deve bilanciare robustezza desiderata, risorse disponibili e vincoli applicativi.

4.6 Analisi Qualitativa e Interpretabilità

Oltre alle metriche quantitative, un'analisi qualitativa fornisce insights su come i modelli prendono decisioni e perché gli attacchi adversariali hanno successo.

Per comprendere come gli attacchi ingannino il modello, la Figura 4.8 presenta un'analisi visiva dello stesso campione (un gatto) in tre condizioni. L'uso di Grad-CAM permette di visualizzare i centri di attenzione del modello:

- **Scenario Pulito:** Il modello classifica correttamente il "gatto" (confidenza 0.987) con mappe di attenzione concentrate correttamente sul volto e sul corpo dell'animale.
- **Sotto Attacco FGSM:** La perturbazione induce un errore (predizione: "frog"). La confidenza cala significativamente (0.432) e le mappe di attenzione appaiono frammentate, segno che il modello sta estraendo feature non coerenti.
- **Sotto Attacco PGD:** L'attacco forza una predizione "dog" con confidenza massima (1.000). Qui il meccanismo di spiegazione è totalmente corrotto: il modello mostra un'attivazione ampia ed estesa, ma basata su segnali puramente adversariali.

Infine, l'analisi statistica delle confidence scores rivela pattern fondamentali sulla calibrazione dei modelli.

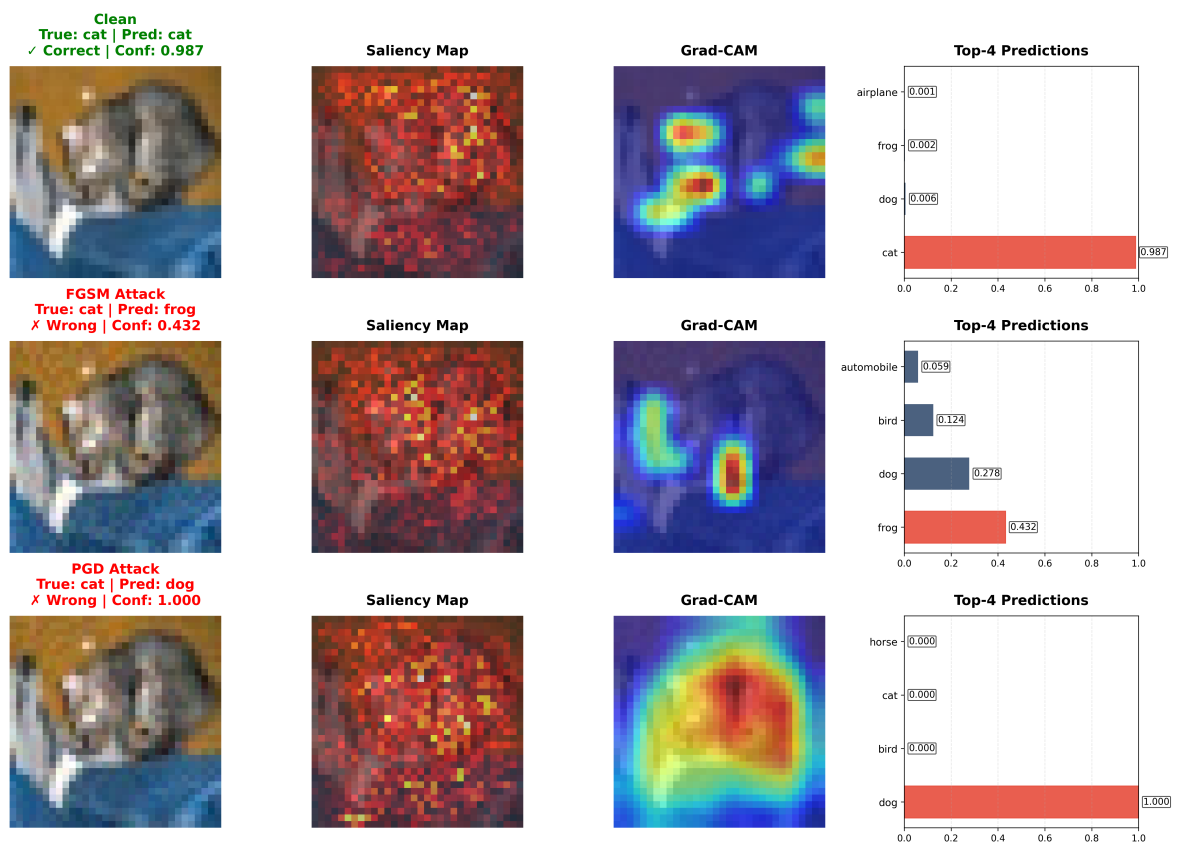


Figura 4.8: Analisi di interpretabilità su un campione di classe "cat".

- **Overconfidence del modello clean:** sotto attacco PGD, il modello non robusto è confidentemente sbagliato, con una mediana di confidenza pari a 1.0.
- **Migliore calibrazione dei modelli robusti:** i modelli PGD-trained e FGSM-trained, pur sotto attacco PGD, mantengono una confidenza mediana intorno a 0.6. Questa riduzione della sicurezza è un segnale di maggiore affidabilità.
- **Strategie di Rejection:** la distribuzione delle confidence rende possibile l'implementazione di filtri basati su soglia (ad esempio $\text{confidence} < 0.5$) per scartare predizioni potenzialmente soggette ad attacco.

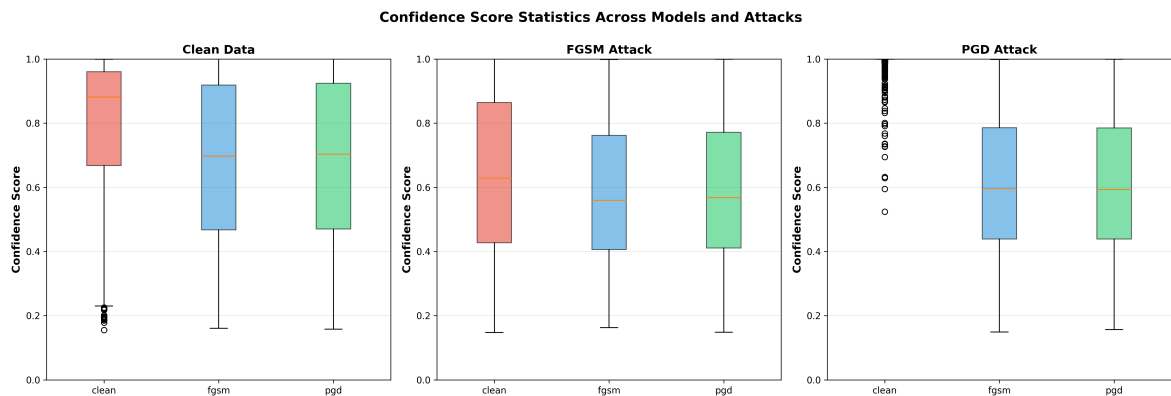


Figura 4.9: Distribuzione delle confidence scores (box plot).

Una sintesi visiva di queste osservazioni è riportata in Figura 4.9.

🔍 Finding 11. Il modello PGD-trained commette errori più semanticamente plausibili (ad esempio confonde classi visivamente simili) invece di produrre predizioni arbitrarie come il modello clean sotto attacco.

🔍 Finding 12. Il modello PGD-trained mostra una migliore calibrazione delle confidence: le predizioni errate presentano confidence significativamente più bassa, permettendo strategie di rejection più efficaci in contesti applicativi.

🔍 Finding 13. Le mappe di attenzione (Grad-CAM/Saliency) rivelano che gli attacchi avversariali operano deviando il fuoco del modello dall'oggetto principale verso feature spurie o verso lo sfondo. Questo spostamento distruttivo dell'attenzione spiega gli errori ad alta confidence osservati nei modelli non robusti.

🗨 **Take-away Message #6.** L’analisi qualitativa conferma che la robustezza avversariale non è solo una questione di metriche quantitative. I modelli robusti processano le immagini in modo qualitativamente diverso, producendo errori più ragionevoli, confidence meglio calibrate e, come mostrano le mappe di attenzione, mantenendo un focus più stabile sulle feature semanticamente rilevanti anche in presenza di perturbazioni.

4.7 Risultati e Risposte: Il Quadro Completo

Questo capitolo ha presentato un’analisi multidimensionale delle prestazioni dei tre modelli. La Tabella 4.9 sintetizza i risultati quantitativi principali che rispondono alle cinque domande di ricerca formulate nel Capitolo 1.

Tabella 4.9: Sintesi completa dei risultati sperimentali.

Modello	Clean Acc (%)	FGSM Acc (%)	PGD Acc (%)	Training Time	Robust Gap (%)	Trade-off Ratio
Clean	86.91	1.41	0.00	40 min	86.91	—
FGSM-trained	79.39	39.01	30.64	3.5 h	48.75	4.07
PGD-trained	78.98	40.75	34.03	12 h	44.95	4.29

Tre evidenze empiriche chiave emergono dall’analisi sperimentale:

1. I modelli CNN standard mostrano vulnerabilità catastrofica: l’accuratezza crolla da 87% a valori prossimi allo zero sotto perturbazioni impercettibili ($\epsilon = 0.03$).
2. L’adversarial training è efficace ma ha un prezzo: guadagna 30-40 punti di robust accuracy sacrificando 7-8 punti di clean accuracy, con overhead computazionale di 2.5-6×.
3. PGD training conferisce robustezza superiore (+11% relativo vs FGSM training), giustificando il maggior costo per applicazioni critiche.

Il Capitolo 5 contestualizza questi risultati rispondendo sistematicamente alle domande di ricerca, discutendo implicazioni pratiche per il deployment, riconoscendo limitazioni dello studio e proponendo direzioni future.

Conclusioni: Verso sistemi di ML robusti e affidabili

Questa tesi ha esplorato il paradosso fondamentale dei moderni sistemi di visione artificiale: modelli estremamente accurati in condizioni controllate che possono fallire catastroficamente di fronte a perturbazioni impercettibili. Attraverso un'analisi empirica sistematica su CIFAR-10, abbiamo non solo quantificato questa vulnerabilità, ma soprattutto mappato il percorso verso possibili soluzioni. I nostri esperimenti hanno rivelato una verità scomoda ma innegabile: l'accuratezza su dati puliti è una metrica ingannevole della reale affidabilità di un modello. Il nostro baseline, performante all'87% in condizioni normali, è crollato a uno sconcertante 0% sotto attacchi avversariali mirati. Questo gap non è un difetto minore, ma una fragilità strutturale che solleva seri interrogativi sull'effettiva "intelligenza" di questi sistemi. La buona notizia emersa dalla ricerca è che l'adversarial training funziona, trasformando modelli vulnerabili in sistemi parzialmente robusti. I nostri modelli protetti hanno mantenuto prestazioni accettabili (30-40%) anche sotto attacchi forti, dimostrando che la robustezza non è un'utopia teorica. Tuttavia, questo miglioramento ha un prezzo: un trade-off con l'accuratezza standard (circa -8%) e un costo computazionale significativo (fino a 6× più lento). Forse il risultato più interessante riguarda come cambia il "pensiero" dei modelli robusti. La nostra analisi qualitativa ha mostrato che, mentre i modelli standard sotto attacco producono errori arbitrari e inspiegabili, i

modelli robustizzati commettono errori semanticamente plausibili (confondere un gatto con un cane) e mantengono un'attenzione visiva più coerente con il contenuto reale dell'immagine.

5.1 Risposte alle Domande di Ricerca

Le evidenze empiriche raccolte nel Capitolo 4 permettono di fornire risposte definitive alle cinque domande che hanno guidato questa ricerca:

🔗 **Answer to RQ₁.** I modelli CNN standard sono estremamente vulnerabili agli attacchi avversariali. Perturbazioni con budget $\epsilon = 0.03$ causano un degrado delle prestazioni da 87% a valori prossimi allo zero (1.41% per FGSM, 0.0% per PGD), dimostrando una fragilità intrinseca che non è evidente dalle prestazioni standard.

🔗 **Answer to RQ₂.** L'adversarial training è una difesa efficace contro gli attacchi avversariali. Esporre il modello a esempi perturbati durante il training aumenta la robust accuracy da valori prossimi allo zero a circa 30–40%, a seconda dell'attacco utilizzato. Questo guadagno è consistente e significativo, confermando l'adversarial training come strategia di riferimento per la robustezza.

🔗 **Answer to RQ₃.** Esiste un trade-off inevitabile tra accuratezza standard e robustezza avversariale. Guadagnare robustezza costa circa 7–8 punti percentuali di accuratezza su dati puliti, ma il rapporto di scambio è favorevole ($\sim 4:1$). Questo compromesso appare intrinseco con le tecniche attuali, sebbene ricerche future possano mitigarlo.

🔗 **Answer to RQ₄.** Il PGD training conferisce robustezza superiore e più generale rispetto al FGSM training. Un modello addestrato con l'attacco più forte (PGD) resiste efficacemente anche ad attacchi più deboli (FGSM), confermando il principio di robustezza discendente. Il maggior costo computazionale ($3.4\times$) è giustificato per applicazioni critiche.

🔗 **Answer to RQ₅.** L’adversarial training comporta un overhead computazionale di 5–18× rispetto al training standard. Questo costo è gestibile con GPU moderne per dataset come CIFAR-10, ma rappresenta una barriera significativa per deployment su larga scala o applicazioni real-time. La scelta della strategia di adversarial training deve bilanciare robustezza desiderata, risorse disponibili e vincoli applicativi.

5.2 Implicazioni Pratiche

I risultati di questa ricerca hanno implicazioni concrete per diverse figure professionali:

- **Per i ricercatori**, i nostri risultati confermano che l’adversarial training con PGD rimane lo standard di riferimento, ma evidenziano l’urgente necessità di metodi più efficienti ed efficaci. Il persistente trade-off tra accuratezza e robustezza rappresenta una delle sfide teoriche più interessanti del campo: non è chiaro se sia un limite fondamentale o se nuove architetture possano aggirarlo.
- **Per gli ingegneri ML**, i risultati forniscono una guida pratica chiara:
 - **Applicazioni critiche (medicina, automotive):** l’investimento in PGD training è non negoziabile
 - **Applicazioni consumer:** FGSM training offre un buon compromesso (+35% robust accuracy, overhead 2.5× invece di 6×)
 - **Regola generale:** mai considerare un modello “pronto per deployment” senza test adversariali

Per i responsabili di prodotto, lo studio sottolinea l’importanza di includere considerazioni sulla robustezza fin dalle prime fasi di design. Il costo aggiuntivo dell’adversarial training deve essere visto non come una spesa opzionale, ma come un’assicurazione necessaria contro fallimenti potenzialmente catastrofici.

5.3 Limitazioni dello Studio

Come ogni studio empirico, anche questo lavoro presenta limiti che circoscrivono la generalizzabilità dei risultati:

- **Dataset e architettura:** L'uso di CIFAR-10 (32×32, 10 classi) e di una singola architettura custom limita l'estensione dei risultati a dataset più complessi (ImageNet) o architetture diverse (Vision Transformers). Architetture più grandi potrebbero mitigare il trade-off grazie a maggiore capacità espressiva.
- **Threat model:** L'analisi si è concentrata su attacchi white-box in norma ℓ_∞ . Nel mondo reale, gli attaccanti operano spesso in condizioni black-box o attraverso manipolazioni fisiche (adversarial patches, 3D objects) con vincoli diversi.
- **Risorse computazionali:** L'uso di singola GPU T4 ha limitato il numero di esperimenti, la dimensione dei modelli testabili, e impedito extensive hyperparameter search o multiple runs con seed diversi per calcolare confidence intervals.
- **Gap teoria-pratica:** I modelli non sono stati testati su dati out-of-distribution, attacchi umani reali, o deployment scenarios operativi con vincoli di latency e throughput.

Questi limiti, piuttosto che sminuire i risultati, indicano direzioni chiare per ricerche future.

5.4 Riflessioni Finali

Questa tesi ha misurato concretamente il prezzo della sicurezza nei sistemi di machine learning. Il trade-off che abbiamo osservato tra accuratezza e robustezza riflette una tensione profonda: da un lato, la capacità di cogliere pattern sottili e complessi; dall'altro, la necessità di basarsi su feature stabili e semanticamente fondate. Forse il messaggio più importante che emerge dal nostro lavoro è che la robustezza non è un optional tecnico, ma una proprietà fondamentale di sistemi ML maturi e affidabili. Così come non accetteremmo un ponte progettato solo per resistere alle condizioni meteorologiche medie, non dovremmo accettare sistemi di intelligenza artificiale che falliscono di fronte a perturbazioni minime ma mirate. Il percorso verso sistemi veramente robusti è ancora lungo, ma i risultati presentati in questa tesi mostrano che siamo sulla strada giusta. L'adversarial training, nonostante

i suoi limiti, rappresenta un primo passo fondamentale. Spetta ora alla comunità di ricerca costruire su queste basi, sviluppando metodi più efficienti, architetture più robuste e, soprattutto, una comprensione teorica più profonda di cosa significhi veramente "comprendere" per una macchina. In un mondo sempre più dipendente da decisioni automatizzate, investire nella robustezza non è solo una questione tecnica, ma una responsabilità etica. Questa tesi spera di aver contribuito, sia pur in piccolo, a questo importante obiettivo.

Bibliografia

- [1] K. Eykholt, I. Evtimov, E. Fernandes, B. Li, A. Rahmati, C. Xiao, A. Prakash, T. Kohno, and D. Song, “Robust physical-world attacks on deep learning visual classification,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018. (Citato a pagina 1)
- [2] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778. (Citato alle pagine 1, 15 e 16)
- [3] A. Krizhevsky and G. Hinton, “Learning multiple layers of features from tiny images,” University of Toronto, Tech. Rep., 2009. (Citato alle pagine 2, 9 e 19)
- [4] I. J. Goodfellow, J. Shlens, and C. Szegedy, “Explaining and harnessing adversarial examples,” in *International Conference on Learning Representations (ICLR)*, 2015. (Citato alle pagine 2, 3, 6, 9, 26, 27, 33 e 41)
- [5] A. Madry, A. Makelov, L. Schmidt, D. Tsipras, and A. Vladu, “Towards deep learning models resistant to adversarial attacks,” in *International Conference on Learning Representations (ICLR)*, 2018. (Citato alle pagine 3, 6, 7, 23, 24, 25, 29, 33, 34, 37, 41, 56 e 79)

-
- [6] A. Sheykina, F. Palomba, and A. De Lucia, “Adversarial odyssey: A systematic literature review,” *ACM Computing Surveys*, vol. 0, no. 0, pp. 1–35, 2024. (Citato a pagina 4)
- [7] A. Athalye, N. Carlini, and D. Wagner, “Obfuscated gradients give a false sense of security: Circumventing defenses to adversarial examples,” in *International Conference on Machine Learning (ICML)*, 2018, pp. 274–283. (Citato a pagina 4)
- [8] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, “Gradient-based learning applied to document recognition,” *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998. (Citato a pagina 9)
- [9] K. Simonyan and A. Zisserman, “Very deep convolutional networks for large-scale image recognition,” in *International Conference on Learning Representations*, 2014. (Citato alle pagine 10 e 16)
- [10] M. D. Zeiler and R. Fergus, “Visualizing and understanding convolutional networks,” *European conference on computer vision*, pp. 818–833, 2014. (Citato a pagina 11)
- [11] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, “Learning representations by back-propagating errors,” *Nature*, vol. 323, no. 6088, pp. 533–536, 1986. (Citato a pagina 13)
- [12] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” *International Conference on Learning Representations*, 2014. (Citato a pagina 14)
- [13] D.-A. Clevert, T. Unterthiner, and S. Hochreiter, “Fast and accurate deep network learning by exponential linear units (elus),” in *International Conference on Learning Representations*, 2015. (Citato a pagina 15)
- [14] A. Krizhevsky, I. Sutskever, and G. E. Hinton, “Imagenet classification with deep convolutional neural networks,” *Advances in neural information processing systems*, vol. 25, 2012. (Citato a pagina 15)
- [15] C. Szegedy, W. Liu, Y. Jia, P. Sermanet, S. Reed, D. Anguelov, D. Erhan, V. Vanhoucke, and A. Rabinovich, “Going deeper with convolutions,” in *Proceedings of*

- the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2015, pp. 1–9. (Citato a pagina 16)
- [16] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, and N. Houlsby, “An image is worth 16x16 words: Transformers for image recognition at scale,” *arXiv preprint arXiv:2010.11929*, 2020. (Citato a pagina 16)
- [17] S. Ioffe and C. Szegedy, “Batch normalization: Accelerating deep network training by reducing internal covariate shift,” in *Proceedings of the 32nd International Conference on Machine Learning (ICML)*, vol. 37. PMLR, 2015, pp. 448–456. (Citato a pagina 16)
- [18] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, “Dropout: a simple way to prevent neural networks from overfitting,” *The journal of machine learning research*, vol. 15, no. 1, pp. 1929–1958, 2014. (Citato a pagina 18)
- [19] K. He, X. Zhang, S. Ren, and J. Sun, “Spatial pyramid pooling in deep convolutional networks for visual recognition,” in *IEEE transactions on pattern analysis and machine intelligence*, vol. 37, no. 9, 2015, pp. 1904–1916. (Citato a pagina 18)
- [20] C. Szegedy, W. Zaremba, I. Sutskever, J. Bruna, D. Erhan, I. Goodfellow, and R. Fergus, “Intriguing properties of neural networks,” *International Conference on Learning Representations*, 2014. (Citato a pagina 22)
- [21] N. Carlini and D. Wagner, “Towards evaluating the robustness of neural networks,” in *IEEE Symposium on Security and Privacy (SP)*, 2017, pp. 39–57. (Citato alle pagine 23 e 31)
- [22] N. Papernot, P. McDaniel, I. Goodfellow, S. Jha, Z. B. Celik, and A. Swami, “Practical black-box attacks against machine learning,” in *ACM on Asia Conference on Computer and Communications Security*, 2017, pp. 506–519. (Citato alle pagine 24 e 25)
- [23] A. Ilyas, S. Santurkar, D. Tsipras, L. Engstrom, B. Tran, and A. Madry, “Adversarial examples are not bugs, they are features,” *Advances in Neural Information Processing Systems*, vol. 32, 2019. (Citato alle pagine 27 e 37)

-
- [24] S.-M. Moosavi-Dezfooli, A. Fawzi, and P. Frossard, "Deepfool: a simple and accurate method to fool deep neural networks," *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 2574–2582, 2016. (Citato a pagina 31)
- [25] J. Su, D. V. Vargas, and K. Sakurai, "One pixel attack for fooling deep neural networks," *IEEE Transactions on Evolutionary Computation*, vol. 23, no. 5, pp. 828–841, 2019. [Online]. Available: <https://doi.org/10.1109/TEVC.2019.2890858> (Citato a pagina 31)
- [26] S.-M. Moosavi-Dezfooli, A. Fawzi, O. Fawzi, and P. Frossard, "Universal adversarial perturbations," *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 1765–1773, 2017. (Citato a pagina 31)
- [27] T. B. Brown, D. Mané, A. Roy, M. Abadi, and J. Gilmer, "Adversarial patch," *arXiv preprint arXiv:1712.09665*, 2017. (Citato a pagina 31)
- [28] N. Papernot, P. McDaniel, X. Wu, S. Jha, and A. Swami, "Distillation as a defense to adversarial perturbations against deep neural networks," in *IEEE Symposium on Security and Privacy (SP)*, 2016, pp. 582–597. (Citato a pagina 34)
- [29] J. Cohen, E. Rosenfeld, and Z. Kolter, "Certified adversarial robustness via randomized smoothing," in *International Conference on Machine Learning*, 2019, pp. 1310–1320. (Citato a pagina 35)
- [30] D. Tsipras, S. Santurkar, L. Engstrom, A. Turner, and A. Madry, "Robustness may be at odds with accuracy," in *International Conference on Learning Representations*, 2019. (Citato a pagina 37)
- [31] H. Zhang, Y. Yu, J. Jiao, E. Xing, L. El Ghaoui, and M. Jordan, "Theoretical analysis of adversarial learning: A minimax approach," in *Advances in Neural Information Processing Systems*, vol. 32, 2019. (Citato a pagina 38)
- [32] L. Rice, E. Wong, and Z. Kolter, "Overfitting in adversarial training: An empirical study," in *International Conference on Machine Learning*, 2020, pp. 7933–7943. (Citato a pagina 39)

- [33] I. Loshchilov and F. Hutter, “Decoupled weight decay regularization,” in *International Conference on Learning Representations (ICLR)*, 2019. (Citato a pagina 55)
- [34] H. Zhang, M. Cisse, Y. N. Dauphin, and D. Lopez-Paz, “mixup: Beyond empirical risk minimization,” in *International Conference on Learning Representations (ICLR)*, 2018. (Citato a pagina 55)

Ringraziamenti

Questo percorso di studi non è stato semplicemente il mezzo per arrivare a un risultato, ma un viaggio fatto di attimi, emozioni, sofferenze e vittorie. Ogni persona qui citata ha partecipato non solo al raggiungimento di questa vetta, ma alla formazione e allo sviluppo della persona che sono. Sono profondamente grato per ciò e spero che questo traguardo segni solo l'inizio di un viaggio immenso.

Al Prof. Fabio Palomba, per la sua incredibile disponibilità e precisione. Nonostante i suoi mille impegni, è sempre riuscito a revisionare il mio lavoro in tempi rapidissimi, con critiche mirate che mi hanno guidato senza mai limitare la mia libertà di azione. La sua pazienza nel venire a patti con le mie tempistiche è stata per me una lezione di umanità oltre che di professionalità.

Ai miei genitori Robert e Filomena, finanziatori primari (e pazientissimi) di questo cammino. Sono le persone che hanno creduto in me più di me stesso, sostenendomi in ogni decisione e offrendomi un appoggio incondizionato, anche quando le mie scelte hanno preso strade inattese. Spero di avervi resi fieri. Vi voglio un mondo di bene e porto con orgoglio le mie origini. Ogni chiamata serale, ogni messaggio e ogni incoraggiamento, mi hanno dato un sostegno immenso anche nei periodi più bui.

A mia sorella Ina, per essere stata come sempre la mia confidente, psicologa e parrucchiera (non sempre retribuita). Di solito è il fratello maggiore a dover infondere forza e speranza nei minori, mentre tu per me sei sempre stata una certezza e una fonte di ispirazione. Mi hai accompagnato dal giorno zero e spero lo farai fino all'ultimo. Ne approfitto anche per ringraziare Mariagrazia, ormai la mia seconda sorella acquisita. È sempre stata un'isola di pace tornare a casa dopo pesanti settimane e vedervi con la macchina pronte per scarrozzarmi a cena e chiacchierare fino a tardi.

Ai miei nonni Rocco e Maddalena, Luigi e Pasqualina. Ai primi due sono grato ogni giorno di avervi qui con me e di potervi vivere a pieno; agli ultimi, so che stasera le stelle in cielo avranno una brillantezza maggiore. Mi avete forgiato e cresciuto per diventare l'uomo che sono oggi. Spero sempre di vivere all'altezza delle vostre virtù e dei vostri insegnamenti. Conserverò gelosamente ogni attenzione che mi avete donato.

Ai miei zii, sguardo silenzioso nella mia vita: il vostro affetto, però, è sempre stato sincero e palpabile. In particolare, ci tengo a ringraziare Zia Stefania, una persona con cui avrò sempre un rapporto profondo e speciale, alla quale avevo promesso questo traguardo. Ce l'abbiamo fatta.

Ai miei cugini, in particolare Cecco, Alessia, Floriana e Alex. Mi avete spronato a studiare e a dare un occhio alla mia carriera accademica anche quando la mia testa era altrove, sempre con affetto e leggerezza.

Alla mia ragazza Emilia. Non stiamo insieme da molto e, purtroppo o per fortuna, non hai assistito a tutto il mio percorso. Solo io, però, conosco il legame che ci contraddistingue e quanto tu mi abbia aiutato in prima persona e sostenuto lungo i mesi più difficili in vista del traguardo. Avrai per sempre una parte importante del mio cuore.

Al mio gruppo stretto, coloro che hanno visto il mio punto più basso e sono rimasti così tanto da potermi anche vedere brillare. Nicolò e Giggino, ci conosciamo da una vita e, anche se a volte il nostro rapporto si allontana per varie vicissitudini, siete sempre lì per me quando conta, e sono grato di ciò. Ogni buon soldato ha una spalla alla sua stregua, ed è qui che entra in gioco Luigi: mille pagine si potrebbero scrivere sulle nostre avventure e sulle conversazioni e progetti che faremo. *Dulcis in fundo*, Francesco, ormai mio convivente e uno dei punti di svolta più importanti. Nonostante le nostre diatribe per l'immondizia in casa, abbiamo condiviso qualunque cosa, ed è qui che si vede la fratellanza. Siete i fratelli che non ho mai avuto.

Al mio gruppo universitario dei Pastafariani: senza di voi non sarei mai arrivato a oggi e il mondo accademico sarebbe rimasto un enorme mostro imbattibile. Inizierei da Giovanni, con il quale abbiamo raggiunto un quantitativo di risate e di disturbo della quiete pubblica imbarazzante, ma al contempo abbiamo macinato ore su ore in biblioteca. Lucone, il mio zio americano nonché coinquilino, mi hai guidato sia con la tua esperienza che con la tua scelleratezza. Wloda, mio fratello polacco, abbiamo condiviso qualunque tipo di emozione, dalla più alta alla più infima; spero che faremo grandi cose. Vinz, mio collega, dal giorno zero ho avuto la conferma di che persona incredibile tu sia e di quel pizzico di follia di cui abbiamo

bisogno. Davidone, la tua gentilezza è rara a questo mondo e passerei volentieri altre mille giornate con te in aula studio. Simone e Salvo, vi sarò per sempre debitore: in un ambiente a volte difficile, siete stati la luce della mia informazione, sempre pronti a dare una mano e a condividere appunti senza chiedere nulla in cambio. Ernesto, un uomo pieno di risorse con cui ho condiviso momenti speciali. A tutti gli altri nomi in questa lista, a prescindere dall'approfondimento, siete stati incredibili: quindi grazie anche a voi Xu, Waluigi, Nazaro, Angelo, Eros, Felice, Carmine, Gianluca, Marco, Orlando e Soemia.

Al mio secondo gruppo. Gianlucone, ci siamo incontrati molto tardi, altrimenti insieme avremmo conquistato il mondo, dal punto di vista pratico e filosofico. Alessandro, il "Comparrone", un uomo che tutti dovrebbero avere nella propria vita, ma per colpa del quale a volte ho "compà" come intercalare. Chiara, non servono parole extra e, per quanto mi riguarda, sei già un pezzo della mia famiglia. Luisa, esistono poche persone al mondo così amorevoli e affettuose. Alessandra, nonostante le incomprensioni, sono felice dei momenti trascorsi insieme e della tua precisione. Domenico e Alessandro, anche se vi vedo poco, sono sempre pieno di gioia quando ci siete.

A Palazzo Principe. La dimora degli ultimi anni delle mie avventure, sinonimo di indipendenza e crescita. I miei coinquilini sono le persone che la rendono speciale: Michele, sempre imperturbabile ogni volta che salgo e scendo quelle scale, un veterano con una buona parola da condividere; Raffaele, "il Matematico", un uomo imprescindibile per la contabilità della casa, con il quale ho fatto chiacchiere davvero stimolanti in questi anni; Carmine, non so quanta pazienza tu abbia, ma non siamo riusciti a farla finire, il coinquilino del piano di sotto che tutti vorrebbero avere. E i libanesi, Giorgio ed Elias: vi voglio bene e spero di andare a caccia con voi prima o poi in Libano; non dimenticherò mai il vostro shawarma.

Al gruppo dei ragazzi di informatica. Inizierei da Caterina, una donna gentilissima e sempre a disposizione per aiutare. Giovio (non è un errore di battitura, ma un difetto di pronuncia che lo renderà per sempre unico). Ludovica, pagherei per rovinare un altro tuo tutorato con la nostra presenza. Francesca, poche parole e molti fatti, una donna essenziale. Manuel, probabilmente questa laurea è dovuta anche a qualche "tarocco" che ho pescato nelle nostre sessioni. Mi avete fatto ricredere sul mondo accademico; persone come voi sono rare e non scorderò mai i momenti vissuti insieme.

Alle ragazze di Mercato. Marinella, qualunque cosa succeda, non smettere mai di nutrire la bambina interiore che ti ha sempre contraddistinta. Martina, la quantità di cibo che mi hai cucinato in questi anni è spropositata, e apprezzo tutte le chiacchiere fatte insieme. Sara, il tuo umorismo e il tuo gusto musicale saranno sempre insostituibili. Non scorderò mai le

mille cene e avventure passate insieme in discoteca; spero ce ne saranno altre mille.

Ai miei amici. Marco, il mio gioielliere di fiducia, l'uomo da cui recarsi quando le cose vanno male e hai bisogno di un tè e una buona chiacchiera. Domenico, le nostre discussioni distilleranno sempre saggezza e ambizione per il futuro che ci attende. Andrea, o come ci piace chiamarlo "Patatino": abbiamo vissuto più disavventure che altro, ma a volte sono le scelte sbagliate che danno vita alle storie migliori. Giorgio, siamo stati poco a contatto per via dei tuoi studi, ma ogni sera passata insieme è stata magica. Sabatino, una delle persone più per bene che abbia mai conosciuto; non vedo l'ora di fare un altro calcetto. Elisa e Dario, siete state le prime persone con cui ho stretto un vero legame da fuorisede e, seppur per pochi mesi, mi avete regalato una leggerezza e una connessione che ricorderò per sempre. Un ringraziamento speciale ai gemelli Nicola e Francesco, Marco e Michele, per avermi fatto ambientare il primo anno di università in un mondo per me alieno, con la loro spontaneità. Grazie anche ai fisici Pasquale, Vincenzo e Francesco "Sinner" per essere stati parte integrante del mio ultimo anno. Ci tengo anche a ringraziare l'associazione Coscienze con i due Leonardo, che ci ha permesso di approfondire amicizie, creare connessioni e bere quantità astronomiche di caffè ogni mattina.

Alle persone che non posso ringraziare, per via di screzi, diatribe e divergenze di cuore. Nonostante ciò, è anche grazie a loro che ho continuato questo percorso, nel bene o nel male. Ogni parola di conforto, ogni battuta, ogni sorriso mi ha dato la forza di arrivare fin qui.

Questa tesi ha contribuito a piantare un albero in Ecuador tramite il progetto Treedom.

<https://www.treedom.net/it/user/sesalab/event/sesa-random-forest>